



**DESIGN AND IMPLEMENTATION OF A WEB-BASED FOOTBALL
LEAGUE MANAGEMENT AND INFORMATION PORTAL FOR INTERNAL
STUDENT FOOTBALL COMPETITIONS (A CASE STUDY OF BAMIDELE
OLUMILUA UNIVERSITY OF EDUCATION, SCIENCE AND
TECHNOLOGY, IKERE-EKITI)**

BY

[STUDENT FULL NAME]

[MATRIC NUMBER]

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE,
FACULTY OF SCIENCE, BAMIDELE OLUMILUA UNIVERSITY OF
EDUCATION, SCIENCE AND TECHNOLOGY, IKERE-EKITI, EKITI STATE,
NIGERIA

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF
THE DEGREE OF BACHELOR OF SCIENCE (B.Sc.) IN COMPUTER SCIENCE

SEPTEMBER, 2026

CERTIFICATION

This is to certify that this project, titled “*Design and Implementation of a Web-Based Football League Management and Information Portal for Internal Student Football Competitions (A Case Study of Bamidele Olumilua University of Education, Science and Technology, Ikere-Ekiti)*”, was carried out by **[STUDENT FULL NAME]** with matriculation number **[MATRIC NUMBER]** in the Department of Computer Science, Faculty of Science, Bamidele Olumilua University of Education, Science and Technology, Ikere-Ekiti, under my supervision.

[SUPERVISOR'S NAME AND TITLE]

Project Supervisor

Date

[HEAD OF DEPARTMENT'S NAME AND TITLE]

Head of Department

Date

External Examiner

Date

DEDICATION

[This project is dedicated to ... — write your personal dedication here, for example to
God Almighty and to your parents or guardians.]

ACKNOWLEDGEMENTS

[Write your personal acknowledgements here. Conventionally, the student thanks God, the project supervisor ([SUPERVISOR'S NAME AND TITLE]) for guidance throughout the work, the Head of Department and lecturers of the Department of Computer Science, family members for their support, and colleagues and friends — including the players, coaches and organisers of the internal football teams — who contributed information during the study.]

ABSTRACT

Internal football competitions between student clubs are an important part of campus life at Bamidele Olumilua University of Education, Science and Technology (BOUESTI), Ikere-Ekiti. Information about these competitions — fixtures, results, league standings, squads and club news — is, however, largely circulated informally, which makes it difficult for students to follow the competition and for organisers to keep accurate, verifiable records. This project designed and implemented a web-based football league management and information portal for the internal student football teams of the university. The system was developed with the Laravel 12 framework (PHP 8.2) following the Model-View-Controller (MVC) architectural pattern, a MariaDB/MySQL relational database, Blade templates, Tailwind CSS and Alpine.js, and was engineered using an iterative and incremental development methodology. It provides a responsive public website where visitors can view fixtures and results, a league table that is computed automatically from completed matches, team and player profiles with statistics derived from recorded match events, a match centre with minute-by-minute events and line-ups, news, a photo gallery and videos; students can also apply for football trials and send enquiries online. A role-protected administration panel allows authorised officials to manage seasons, competitions, teams, players, fixtures, results, match events, line-ups, news (through a sanitised rich-text editor), media, staff, trial applications and contact messages, while a CSV import tool supports bulk loading of real squad and fixture data. The system was verified with 91 automated feature and unit tests (901 assertions) covering the core workflow, validation, security, data integrity and rendering, and was checked for responsiveness at ten screen widths from 320 to 1440 pixels. The results show that the portal eliminates manual computation of standings, preserves historical match data, protects students' private information, and gives the university football community a single, reliable and accessible source of information.

Keywords: football league management, web application, Laravel, MVC, relational database, sports information system, responsive web design.

TABLE OF CONTENTS

CERTIFICATION	ii
DEDICATION	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT	v
LIST OF TABLES	ix
LIST OF FIGURES	x
LIST OF ABBREVIATIONS	xi
CHAPTER ONE: INTRODUCTION	1
1.1 Background to the Study	1
1.2 Statement of the Problem	1
1.3 Aim and Objectives of the Study	2
1.4 Scope of the Study	3
1.5 Significance of the Study	3
1.6 Limitations of the Study	3
1.7 Definition of Terms	4
1.8 Organisation of the Report	4
CHAPTER TWO: LITERATURE REVIEW	5
2.1 Introduction	5
2.2 Information Systems	5
2.3 Sports Information and League Management Systems	5
2.4 Web Applications and the Client-Server Model	6
2.5 The Model-View-Controller (MVC) Architecture	6
2.6 Web Development Frameworks and Laravel	7
2.7 Relational Databases	7
2.8 Responsive and Accessible Web Design	7
2.9 Web Application Security	8
2.10 Software Development Methodologies	8
2.11 Software Testing	8
2.12 Review of Existing Approaches and Research Gap	9

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN	10
3.1 Introduction	10
3.2 Research and Development Methodology	10
3.3 Analysis of the Existing System	10
3.4 The Proposed System	11
3.5 Requirements Specification	12
3.6 System Architecture	13
3.7 Use Case Model	14
3.8 Data Flow	15
3.9 Database Design	15
3.10 Algorithm Design	19
3.11 Input and Output Design	21
3.12 Security Design	21
CHAPTER FOUR: SYSTEM IMPLEMENTATION AND TESTING	22
4.1 Introduction	22
4.2 Implementation Environment	22
4.3 System Structure and Modules	22
4.4 Test Data	23
4.5 Implementation Screenshots	23
4.6 System Testing	32
4.7 Performance Considerations	34
4.8 Deployment	34
CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS . .	35
5.1 Summary	35
5.2 Achievement of Objectives	35
5.3 Conclusion	36
5.4 Recommendations	36
5.5 Suggestions for Further Work	36
REFERENCES	37
APPENDIX A: USER MANUAL	38
APPENDIX B: INSTALLATION AND DEPLOYMENT GUIDE	40

APPENDIX C: SELECTED SOURCE CODE	41
---	-----------

LIST OF TABLES

Table 0.1: Abbreviations used in this report	xi
Table 1.1: Definition of terms	4
Table 2.1: Comparison of approaches to managing internal competition information	9
Table 3.1: Development increments	10
Table 3.2: Technologies selected and justification	11
Table 3.3: Functional requirements	12
Table 3.4: Non-functional requirements	12
Table 3.5: Use case description — Record a match result	14
Table 3.6: seasons	16
Table 3.7: competitions	17
Table 3.8: teams	17
Table 3.9: players	17
Table 3.10: fixtures	18
Table 3.11: match_events	18
Table 3.12: fixture_players (line-ups)	19
Table 3.13: Content and communication tables (summary)	19
Table 3.14: Security measures	21
Table 4.1: Development and runtime environment	22
Table 4.2: Main modules	22
Table 4.3: Summary of test cases and results	32

LIST OF FIGURES

Figure 3.1: System architecture	13
Figure 3.2: Use case diagram	14
Figure 3.3: Context diagram (level-0 data flow diagram)	15
Figure 3.4: Entity-relationship diagram	16
Figure 3.5: Flowchart of the league table algorithm	20
Figure 4.1: Homepage	24
Figure 4.2: League table with form guide, computed from completed fixtures	24
Figure 4.3: Match centre with score and minute-by-minute events	25
Figure 4.4: Player profile (public details only; matriculation number is never shown)	25
Figure 4.5: Teams directory with generated crests	26
Figure 4.6: Squad page with team and position filters	26
Figure 4.7: Fixtures with competition, team and month filters	27
Figure 4.8: Match report article	27
Figure 4.9: Online trial application form	28
Figure 4.10: Mobile views — homepage, league table and navigation menu	29
Figure 4.11: Administration dashboard	29
Figure 4.12: Match management — result entry and match events	30
Figure 4.13: Player management with search and filters (admin-only matriculation numbers)	30
Figure 4.14: News editor with rich-text toolbar	31
Figure 4.15: Trial applications inbox	31

LIST OF ABBREVIATIONS

Table 0.1: Abbreviations used in this report

Abbreviation	Meaning
BOUESTI	Bamidele Olumilua University of Education, Science and Technology
CRUD	Create, Read, Update and Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
GD	Graphics Draw (PHP image library)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
MVC	Model-View-Controller
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PHP	PHP: Hypertext Preprocessor
RDBMS	Relational Database Management System
SQL	Structured Query Language
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Football is the most widely played and followed sport in Nigeria, and universities are no exception. Beyond inter-university competitions, many Nigerian universities host internal competitions in which student-organised clubs — often formed around departments, faculties, halls of residence or groups of friends — compete against each other throughout the academic session. These competitions promote physical fitness, discipline, teamwork and social cohesion, and they provide a pathway for talented students to be identified for the institution's representative teams.

At Bamidele Olumilua University of Education, Science and Technology (BOUESTI), Ikere-Ekiti, several student football clubs compete internally. The clubs covered by this study are Amapro FC, Elite FC, Young Boys FC, CSC Elites, Engines Boys, Sovereignty FC and AMCOMS. A competition of this kind produces a steady stream of information: fixture schedules, venues and kick-off times, match results, goal scorers and disciplinary records, league standings, squad lists, trial announcements and club news.

Modern web technologies make it possible to manage this information in a structured database and to publish it instantly to any device with a browser. Professional football competitions routinely provide official websites with live standings, match centres and player statistics. The same capability, adapted to the scale and needs of a university, can transform how internal competitions are organised and followed. This project applies established software engineering practice — requirements analysis, relational database design, the Model-View-Controller architecture and automated testing — to build such a platform for BOUESTI's internal football competitions.

1.2 Statement of the Problem

Information about internal football competitions is typically managed and shared informally — through notice boards, word of mouth, messaging-app groups and individual record books kept by organisers. This approach gives rise to the following problems:

1. **Fragmented and inaccessible information.** There is no single, authoritative place where a student can see the fixture list, the latest results, the league table or a team's squad. Messages in chat groups are quickly buried and are not accessible to

students outside those groups.

2. **Error-prone manual computation of standings.** League tables computed by hand from paper records are slow to update and prone to arithmetic errors in points, goal difference and ordering, which can lead to disputes.
3. **Poor record keeping and loss of history.** Results, scorers and disciplinary records of previous matches and seasons are easily lost, so player achievements and club history cannot be verified.
4. **No recognition of individual players.** Without a structured record of match events, individual statistics such as goals, assists and appearances cannot be produced, and talented players receive little visibility.
5. **Inefficient recruitment and communication.** Students who want to join a team have no formal channel to apply for trials, and enquiries to organisers are handled ad hoc.
6. **Privacy risks.** When student information is shared informally, sensitive details such as matriculation numbers and phone numbers may be exposed to people who should not see them.

1.3 Aim and Objectives of the Study

The **aim** of this project is to design and implement a secure, responsive, web-based football league management and information portal for the internal student football competitions of BOUESTI.

The specific **objectives** are to:

1. analyse the information requirements of the internal football competition and design a normalised relational database for seasons, competitions, teams, players, fixtures, match events, line-ups, news, media, staff, trial applications and enquiries;
2. develop a public, mobile-responsive website that presents fixtures, results, a match centre, team and player profiles, news, a gallery and videos;
3. implement an algorithm that computes league standings automatically from completed fixtures, and services that derive player and team statistics from recorded match events;
4. develop a role-based administration panel through which authorised officials manage all competition data and website content;
5. provide online channels for trial applications and contact enquiries, with an administrative inbox;
6. secure the system against common web vulnerabilities and protect students' private data; and

7. test the system with automated tests and evaluate it against the stated requirements.

1.4 Scope of the Study

The study covers the internal football competitions played between student clubs of BOUESTI. The system manages multiple seasons and competitions (league, cup, friendly and tournament types), the seven internal teams and their players, the full match lifecycle (scheduling, live and completed statuses, postponement and cancellation), match events (goals, penalties, own goals, assists, cards and substitutions), line-ups, automatically computed standings and statistics, news, a gallery, videos, management and coaching staff, trial applications and contact messages. Bulk import of real data from spreadsheets and optimisation of uploaded images are included.

The following are outside the scope of this version: live minute-by-minute commentary pushed in real time, ticketing and online payment, fantasy football, betting, player transfers between clubs, native mobile applications and representation of inter-university competitions.

1.5 Significance of the Study

- **Students and supporters** gain a single, always-available source of accurate fixtures, results, standings and news on any device.
- **Players** receive recognition through public profiles and statistics built from verified match records.
- **Competition organisers and the sports unit** save time: results entered once automatically update standings, statistics and the homepage, and historical records are preserved.
- **Aspiring players** have a formal, transparent channel for applying for trials.
- **The university** gains a professional digital presence for campus sport that promotes student engagement and wellbeing.
- **Academically**, the project demonstrates the application of software engineering, database design, web security and testing principles to a real campus problem, and can serve as a reference for similar systems.

1.6 Limitations of the Study

- **Data availability.** Official squad lists, staff details and historical results were not available in machine-readable form during development. The system was therefore demonstrated and tested with a generated, clearly labelled sample season of

fictional players and staff; a CSV import tool is provided so that verified data can be loaded without code changes.

- **Connectivity.** Users need internet access to reach the portal; it does not work offline.
- **Human input.** The accuracy of statistics depends on officials recording results and match events correctly and promptly.
- **Time and resources** limited the project to a web application; features such as push notifications and native mobile apps are recommended for future work.

1.7 Definition of Terms

Table 1.1: Definition of terms

Term	Definition
Fixture	A scheduled match between a home team and an away team in a competition.
Matchday	A round of a competition in which each team plays at most one match.
Standings / league table	The ranking of teams in a competition by points, goal difference and goals scored.
Goal difference (GD)	Goals scored (GF) minus goals conceded (GA).
Match event	A recorded incident in a match: goal, penalty, own goal, assist, yellow or red card, or substitution.
Line-up	The players named for a match: the starting eleven and the substitutes.
Clean sheet	A match in which a team (and its starting goalkeeper) concedes no goals.
Framework	A reusable software platform that provides structure and common services for building applications.
Soft delete	Marking a record as deleted without physically removing it, so that historical references remain valid.
Responsive design	A web design approach in which layouts adapt to the size of the user's screen.

1.8 Organisation of the Report

Chapter One introduces the study. Chapter Two reviews literature on sports information systems, web application architecture, databases, security and usability. Chapter Three presents the analysis of the existing system, the requirements and the design of the proposed system, including its architecture, database and algorithms. Chapter Four describes the implementation, testing and results. Chapter Five summarises the work and presents conclusions and recommendations. The appendices contain a user manual, an installation guide and selected source code.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews the concepts, technologies and existing approaches relevant to the development of a football league management and information portal. It covers information systems and sports information systems, web application architecture and the Model-View-Controller pattern, web frameworks, relational databases, responsive and accessible design, web application security and software testing, and it concludes with a review of existing approaches and the gap this project fills.

2.2 Information Systems

An information system is an organised combination of people, procedures, data, software and hardware that collects, processes, stores and distributes information to support decision making and operations. Sommerville (2016) describes such systems as socio-technical systems in which the software component must be understood in relation to the organisation that uses it. For a competition, the raw data are fixtures, results and match events; the information derived from them — standings, statistics and reports — is what users actually need. A well-designed information system captures each fact once, at its source, and derives all other information from it automatically, which is the principle adopted in this project: officials record results and events once, and the league table and player statistics are computed rather than typed.

2.3 Sports Information and League Management Systems

Sports information systems support the administration of competitions and the publication of information to participants and the public. Their typical functions are: registration of teams and players; generation and publication of fixture schedules; capture of results and match events; computation of standings; production of individual and team statistics; and communication through news and media. At the professional level these functions are provided by official competition websites with match centres, live tables and detailed statistics. At the grassroots and amateur level, commercial software-as-a-service platforms (for example, TeamSnap and LeagueApps) offer scheduling, registration and communication features on a subscription basis.

For a university's internal competitions these general solutions have drawbacks: they are designed around foreign clubs and payment models, they are not branded to the

institution, they do not model student-specific data such as departments, levels and matriculation numbers, and they may store student data on third-party services outside the institution's control. A purpose-built system, owned and hosted by the institution, can reflect the structure of campus competitions exactly and apply appropriate privacy rules.

2.3.1 League standings

Most association football leagues rank teams by awarding three points for a win, one for a draw and none for a defeat, and break ties first by goal difference and then by goals scored. Standings are a pure function of the set of completed results: given the same results, the table is always the same. This property means that standings should not be stored and edited manually but computed from the results, which removes a whole class of inconsistency errors — a design decision central to this project (Section 3.10).

2.4 Web Applications and the Client-Server Model

A web application is software whose user interface runs in a web browser and which communicates with a server over the HyperText Transfer Protocol (HTTP). Fielding (2000) formalised the architectural principles of the web, including the stateless client-server interaction on which web applications are built. Web applications need no installation on the user's device, are updated centrally and are reachable from any device with a browser — properties that make them well suited to publishing competition information to a large and varied student audience.

2.5 The Model-View-Controller (MVC) Architecture

The Model-View-Controller pattern, first described for Smalltalk-80 user interfaces (Krasner and Pope, 1988), separates an application into three responsibilities: the **Model**, which represents data and business rules; the **View**, which presents data to the user; and the **Controller**, which receives user input, coordinates the model and selects the view. This separation of concerns improves maintainability, testability and the ability of different people to work on different parts of the system (Gamma et al., 1994; Fowler, 2002). Server-side web frameworks such as Laravel adopt MVC: HTTP requests are routed to controllers, controllers use models to read and write the database, and responses are rendered from view templates.

2.6 Web Development Frameworks and Laravel

A web framework provides the common infrastructure required by most web applications — routing, request handling, database access, templating, authentication, validation and security protections — so that developers can concentrate on application-specific logic. Laravel is an open-source PHP framework that follows the MVC pattern and provides, among other features, the Eloquent object-relational mapper (an implementation of the Active Record pattern described by Fowler, 2002), database migrations and seeders, the Blade templating engine, form request validation, middleware, authentication scaffolding, file storage abstraction and first-class support for automated testing (Laravel, 2025). These features, together with its large community and documentation, made Laravel the chosen framework for this project (Section 3.4).

2.7 Relational Databases

The relational model, introduced by Codd (1970), represents data as relations (tables) of tuples (rows) with attributes (columns), related to one another through keys. Relational database management systems (RDBMS) such as MySQL and MariaDB implement this model and use Structured Query Language (SQL). Normalisation reduces redundancy and update anomalies by ensuring that each fact is stored once (Elmasri and Navathe, 2016; Connolly and Begg, 2015). Referential integrity, enforced through foreign-key constraints, guarantees that relationships between records remain valid — for example, that every match event belongs to an existing fixture. Competition data is highly structured and relational (a fixture relates two teams; an event relates a fixture, a team and players), which makes an RDBMS the appropriate storage technology.

2.8 Responsive and Accessible Web Design

Students access the web predominantly from mobile phones. Responsive web design, articulated by Marcotte (2010), uses fluid layouts, flexible images and CSS media queries so that a single site adapts to screens of any size. Accessibility ensures that people with disabilities can perceive, operate and understand web content; the World Wide Web Consortium's Web Content Accessibility Guidelines (WCAG) 2.1 define testable criteria such as sufficient colour contrast, keyboard operability, visible focus indicators and text alternatives for images (W3C, 2018). Usability research emphasises consistency, visibility of system status, error prevention and clear error messages (Nielsen, 1993). These principles guided the user-interface work described in Chapter Four.

2.9 Web Application Security

Web applications are exposed to attack from anyone on the internet. The OWASP Top 10 (OWASP Foundation, 2021) lists the most critical web application security risks, including broken access control, injection, cryptographic failures, insecure design and security misconfiguration. Established countermeasures relevant to this project are: role-based access control enforced on the server; parameterised queries (provided by an ORM) to prevent SQL injection; output escaping and HTML sanitisation to prevent cross-site scripting (XSS); anti-forgery tokens to prevent cross-site request forgery (CSRF); secure password hashing; validation of all input, including uploaded files; and rate limiting of public forms. Privacy regulations and good practice additionally require that personal data be collected only for a purpose and exposed only to those who need it.

2.10 Software Development Methodologies

Pressman and Maxim (2020) and Sommerville (2016) describe several software process models. The waterfall model proceeds through requirements, design, implementation, testing and maintenance sequentially; it is simple to manage but inflexible when requirements change. Iterative and incremental models deliver the system in successive increments, each adding functionality and each refined through feedback; agile methods emphasise working software, collaboration with users and responsiveness to change. Because the requirements of this project were refined progressively (the user interface was built first, then the data model, then administration, then supporting features), an iterative and incremental approach was adopted (Section 3.2).

2.11 Software Testing

Testing provides evidence that software meets its requirements and reveals defects. Unit tests verify individual components in isolation; feature (integration) tests verify that components work together — for example, that submitting a form stores a record and changes a page; system and acceptance tests verify the whole system against user requirements (Sommerville, 2016). Automated tests can be re-run after every change to detect regressions, a practice central to test-driven development (Beck, 2002). This project uses the Pest testing framework on top of PHPUnit to automate feature tests that exercise the application through HTTP requests against an isolated test database.

2.12 Review of Existing Approaches and Research Gap

Table 2.1: Comparison of approaches to managing internal competition information

Approach	Strengths	Weaknesses
Notice boards and word of mouth	No cost; familiar	Limited reach; slow; no history; no statistics
Messaging-app groups	Fast; free; widely used	Information buried in chat; restricted membership; no structure; privacy risks
Spreadsheets kept by organisers	Structured; can compute tables	Single user; manual errors; not published; hard to share securely
Commercial league platforms	Rich features; hosted	Subscription cost; not institution-branded; no student-specific data; data held by third parties
Proposed portal (this project)	Single source of truth; automatic standings and statistics; responsive public site; role-based admin; institution-owned; privacy by design	Requires hosting and trained officials to enter data

The review shows that informal approaches cannot provide accurate, accessible and historical information, and general commercial tools do not fit the structure, branding and privacy needs of a university's internal competitions. The gap addressed by this project is a purpose-built, institution-owned system that stores each competition fact once, derives standings and statistics automatically, publishes them on a responsive and accessible website, and protects student data through role-based access control.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter presents the methodology used, an analysis of the existing system, the requirements of the proposed system and its design: the system architecture, use cases, data flow, database design, the key algorithms and the input, output and security design.

3.2 Research and Development Methodology

Requirements were gathered through observation of how internal competition information is currently shared, informal discussion with students involved in the clubs, and a review of the information presented by established football competition websites. The system was built using an **iterative and incremental** development methodology, in which the product was delivered in increments that were each designed, implemented, tested and reviewed before the next began:

Table 3.1: Development increments

Increment	Deliverables
1. User interface	Responsive public website design migrated into Laravel Blade layouts and components; UI/UX and accessibility refinement.
2. Data model	Relational schema (migrations), Eloquent models, relationships, scopes and seeders for seasons, competitions and teams.
3. Services	League table computation, team and player statistics.
4. Administration	Role-protected admin panel with CRUD for all entities, result entry, match events and line-ups.
5. Dynamic public site	Public pages connected to the database; trials and contact forms; empty states; pagination.
6. Supporting features	Rich-text news editor with HTML sanitisation, image optimisation, CSV import, sample-season generator.
Continuous	Automated tests written alongside each increment and re-run after every change.

3.3 Analysis of the Existing System

In the existing arrangement, fixtures are agreed among organisers and announced on notice boards or in messaging groups; results are reported verbally or by message after each match; any league table is computed by hand; and records of scorers and cards, if kept at all, are held privately by individual organisers. Students who wish to join a team approach team officials informally.

3.3.1 Weaknesses of the existing system

- No central, public and permanent record of fixtures, results or standings.
- Manual table computation is slow and error-prone, and corrections are hard to publicise.
- Individual statistics (goals, assists, cards, appearances) are not produced.
- Historical data is lost between sessions.
- No formal channel for trial applications or enquiries.
- Student personal data may be circulated without access control.

3.4 The Proposed System

The proposed system is a web-based portal with two interfaces sharing one database: a **public website** for students and supporters, and an **administration panel** for authorised officials. Officials record each fact once — a player, a fixture, a result, a goal — and the system derives everything else: standings, team and player statistics, the homepage's next match and recent results, and the match centre. The advantages over the existing system are accuracy, availability, preservation of history, recognition of players, and privacy through role-based access control.

3.4.1 Justification of the chosen technologies

Table 3.2: Technologies selected and justification

Layer	Technology	Justification
Server language	PHP 8.2	Mature, widely hosted (including low-cost shared hosting), fast, and taught in Nigerian computing curricula.
Framework	Laravel 12	MVC structure, Eloquent ORM, migrations, validation, authentication, CSRF protection and testing tools out of the box.
Database	MariaDB 10.4 / MySQL 8 (SQLite for tests)	Reliable open-source RDBMS with foreign-key support; bundled with XAMPP for local development.
Public UI	Blade, custom CSS, vanilla JavaScript	Preserves the bespoke BOUESTI design; lightweight for mobile data connections.
Admin UI	Tailwind CSS 3, Alpine.js 3, Vite 7	Utility-first styling and small reactive components; production asset bundling.
Authentication	Laravel Breeze	Secure login, registration, password reset and email verification scaffolding.
Rich text	Trix editor + Symfony HtmlSanitizer	Simple editor for officials; server-side allow-list sanitisation prevents XSS.
Images	PHP GD extension	Resizing and WebP conversion without additional dependencies.

Layer	Technology	Justification
Testing	Pest 3 (on PHPUnit)	Readable automated feature tests against an in-memory database.

3.5 Requirements Specification

3.5.1 Functional requirements

Table 3.3: Functional requirements

ID	Requirement
FR1	The system shall display upcoming fixtures and results, filterable by competition, team and month.
FR2	The system shall compute and display the league table from completed fixtures (3 points for a win, 1 for a draw; ties broken by goal difference then goals scored).
FR3	The system shall display teams, squads grouped by position, and player profiles with appearances, goals, assists, cards and (for goalkeepers) clean sheets.
FR4	The system shall provide a match centre with score, status, minute-by-minute events, line-ups, match summary and related photos.
FR5	The system shall publish news articles, a photo gallery with categories and a video list.
FR6	The system shall accept trial applications and contact messages with validation and spam protection.
FR7	Only authenticated administrators shall access the administration panel.
FR8	Administrators shall create, update and delete seasons, competitions, teams, players, fixtures, news, gallery items, videos and staff.
FR9	Administrators shall enter results and record match events and line-ups; only players of the teams in the fixture may be selected.
FR10	Administrators shall review trial applications and contact messages and set their status.
FR11	The system shall import teams, players, staff, fixtures and events from CSV files, rejecting the whole import if any row is invalid.
FR12	The system shall resize and convert uploaded images and remove their metadata.

3.5.2 Non-functional requirements

Table 3.4: Non-functional requirements

ID	Category	Requirement
NFR1	Usability	Pages shall be usable on screens from 320 px to 1440 px wide without horizontal scrolling; controls shall be at least 44 px high on touch screens.
NFR2	Accessibility	The interface shall follow WCAG 2.1 principles: keyboard access, visible focus, text alternatives, semantic landmarks and adequate contrast.
NFR3	Security	The system shall enforce role-based access, CSRF protection, validation of all input, escaping of output and sanitisation of rich text.
NFR4	Privacy	Matriculation numbers, state of origin and applicants' contact details shall never be shown on public pages.

ID	Category	Requirement
NFR5	Integrity	Deleting teams, players, news or staff shall not destroy historical match records (soft deletes and restrictive foreign keys).
NFR6	Performance	Statistics shall be computed with aggregate queries; images shall be optimised; pages shall load their critical styles without blocking on third-party resources.
NFR7	Maintainability	Code shall follow MVC with services for business logic, form requests for validation and reusable view components.
NFR8	Reliability	Core workflows shall be covered by automated tests that pass before deployment.

3.6 System Architecture

The system follows a three-tier web architecture implemented with Laravel's MVC structure (Figure 3.1). The browser sends HTTP requests; routes and middleware apply authentication, administrator authorisation, CSRF verification and rate limiting; form requests validate input; controllers coordinate the request, delegating business logic to service classes; Eloquent models read and write the relational database; and Blade views render the response. Uploaded files are stored on Laravel's public storage disk.

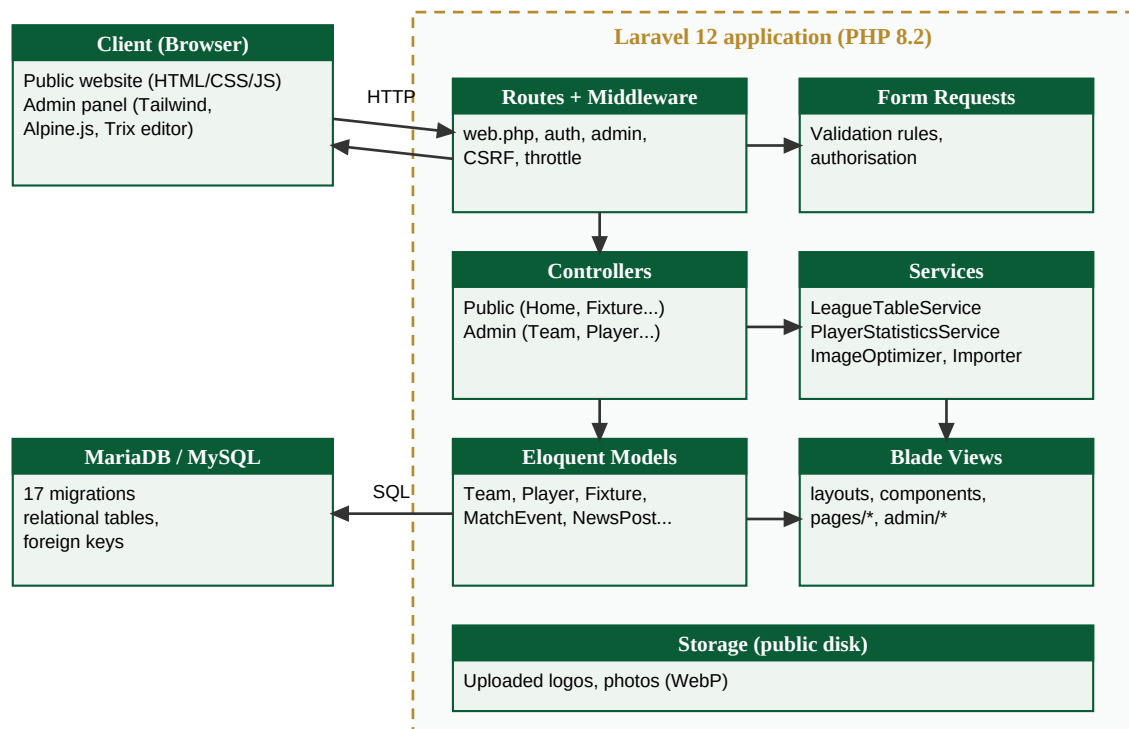


Figure 3.1: System architecture

3.7 Use Case Model

Three actors interact with the system (Figure 3.2). A **Visitor** is anyone browsing the public site. A **Student** is a visitor who submits a trial application or a contact message. An **Administrator** is an authenticated official with the admin role who manages all data. Table 3.5 describes the principal use case.

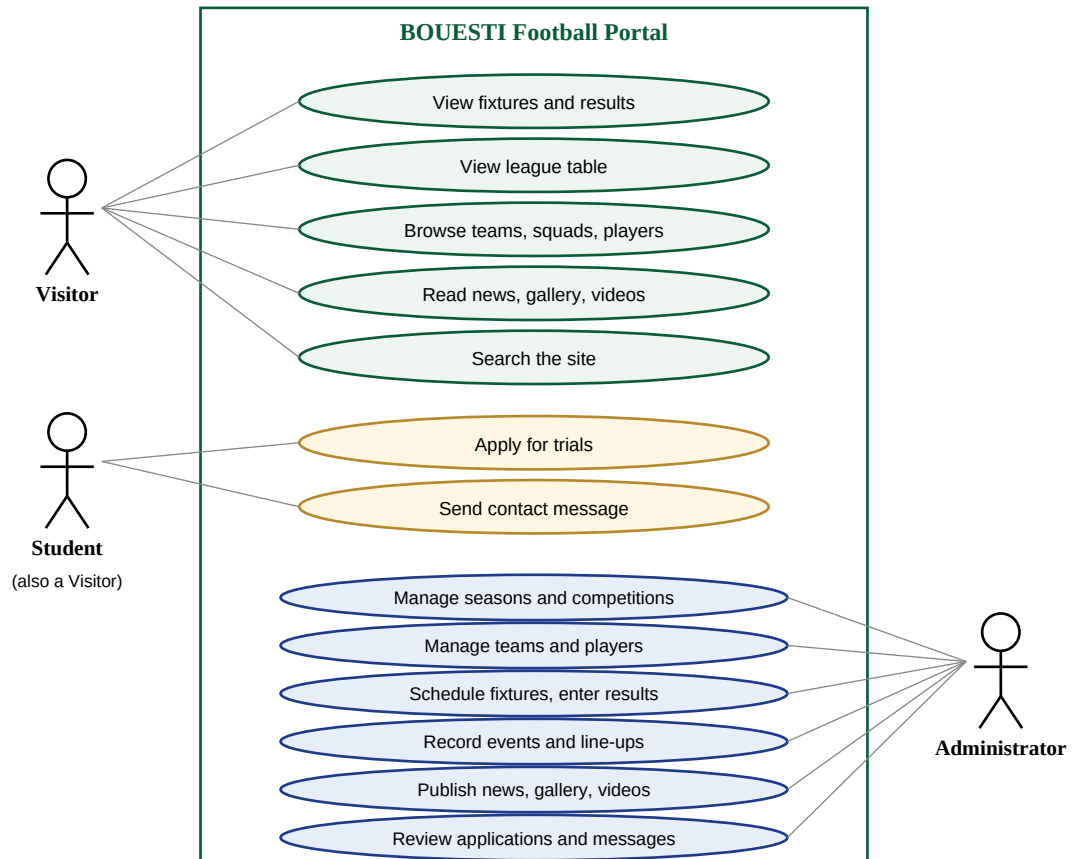


Figure 3.2: Use case diagram

Table 3.5: Use case description — Record a match result

Item	Description
Use case	Record a match result
Actor	Administrator
Pre-condition	The administrator is logged in; the fixture exists.
Main flow	1. Open Fixtures and select the match. 2. Choose status “Full time” and enter the home and away scores. 3. Optionally add the referee, attendance and a short summary. 4. Save. 5. Add goal, assist, card and substitution events; for each, choose the team — the player list shows only that team's players. 6. Tick the line-up for each team.
Alternative flow	Scores missing for a completed match: the system shows a validation message and nothing is saved. A player not in the selected team: the event is rejected.

Item	Description
Post-condition	The league table, team statistics, player statistics, homepage results and match centre reflect the result immediately.

3.8 Data Flow

Figure 3.3 shows the context-level data flow diagram. Visitors and students send requests, trial applications and contact messages and receive published pages; administrators supply competition data and content and receive the dashboard, statistics and inbox; the system optionally notifies the club by email when a contact message arrives.

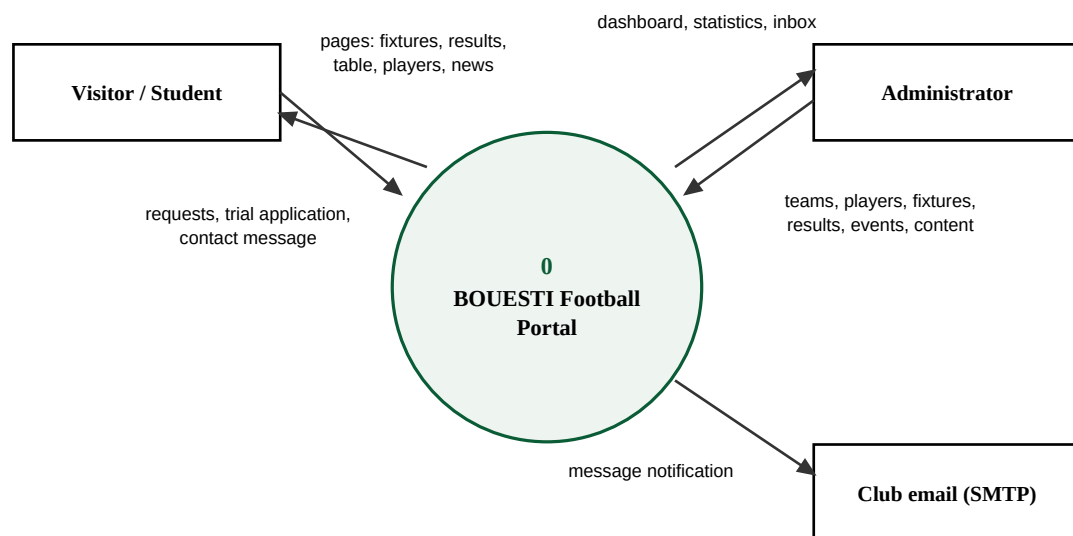
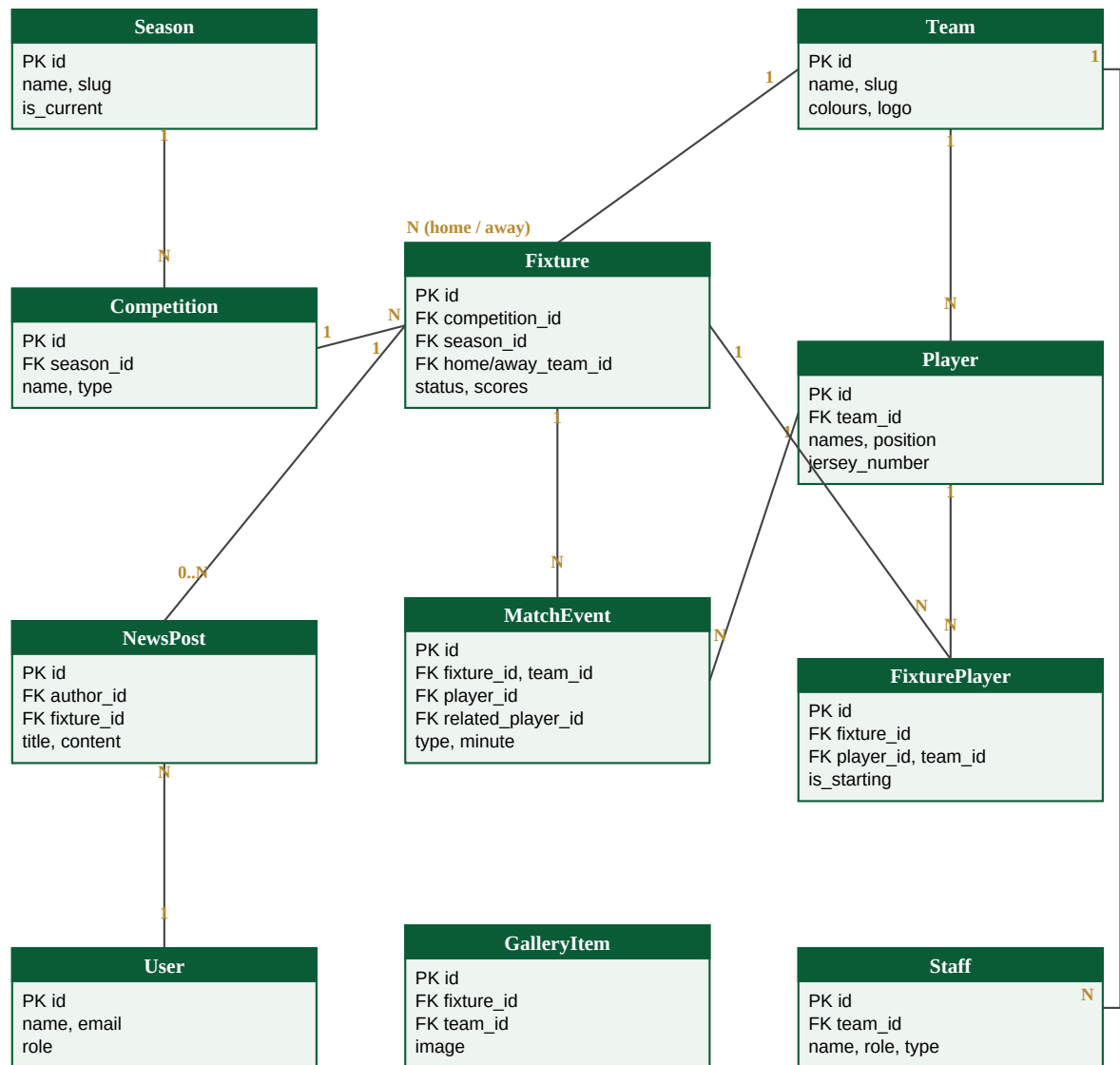


Figure 3.3: Context diagram (level-0 data flow diagram)

3.9 Database Design

The database was designed in third normal form. Each real-world entity has its own table with a surrogate primary key; relationships are enforced by foreign keys. Standings and statistics are **not stored**; they are derived from fixtures and match events, which removes update anomalies. Deletion rules were chosen deliberately: seasons, competitions and teams referenced by fixtures are protected (*restrict*); match events and line-ups are removed with their fixture (*cascade*); optional links such as a gallery photo's fixture are cleared (*set null*); and teams, players, news and staff are soft-deleted so that historical results keep their names. Figure 3.4 shows the entity-relationship diagram.



GalleryItem optionally references Fixture and Team. Stand-alone: videos, trial_applications, contact_messages.

Figure 3.4: Entity-relationship diagram

3.9.1 Data dictionary

Tables 3.6 to 3.13 describe the main tables. Every table also has *created_at* and *updated_at* timestamps; soft-deletable tables have *deleted_at*.

Table 3.6: seasons

Field	Type	Description / constraint
id	BIGINT, PK	Surrogate key

Field	Type	Description / constraint
name	VARCHAR(255)	e.g. 2026/2027
slug	VARCHAR, unique	URL identifier
start_date, end_date	DATE, null	Session dates
is_current	BOOLEAN	Only one season may be current (enforced in the model)
is_active	BOOLEAN	Shown in selectors

Table 3.7: competitions

Field	Type	Description / constraint
id	BIGINT, PK	
season_id	FK to seasons (restrict)	Owning season
name, slug	VARCHAR	Name; unique slug
short_name	VARCHAR(30), null	e.g. BFL
type	VARCHAR(20)	league cup friendly tournament
description, logo	TEXT / VARCHAR, null	
start_date, end_date	DATE, null	
is_active	BOOLEAN	

Table 3.8: teams

Field	Type	Description / constraint
id	BIGINT, PK	
name, slug	VARCHAR; slug unique	e.g. Amapro FC / amapro-fc
short_name	VARCHAR(12), null	Badge text
logo	VARCHAR, null	Path on public disk
primary_color, secondary_color	CHAR(7), null	Hex colours
description	TEXT, null	
founded_year	SMALLINT, null	
captain_name, coach_name	VARCHAR, null	
is_active	BOOLEAN	
deleted_at	TIMESTAMP, null	Soft delete

Table 3.9: players

Field	Type	Description / constraint
id	BIGINT, PK	

Field	Type	Description / constraint
team_id	FK to teams (restrict)	Player's club
first_name, last_name, slug	VARCHAR; slug unique	
photo	VARCHAR, null	
jersey_number	TINYINT, null	1–99; unique within team (validated)
position	VARCHAR(20)	goalkeeper defender midfielder forward
department, level	VARCHAR, null	Public
matric_number, state_of_origin	VARCHAR, null	Private — admin only; hidden from serialisation
dominant_foot, height, bio	VARCHAR / TEXT, null	
is_captain, is_featured, is_active	BOOLEAN	
deleted_at	TIMESTAMP, null	Soft delete

Table 3.10: fixtures

Field	Type	Description / constraint
id	BIGINT, PK	
competition_id, season_id	FK (restrict)	Season always follows the competition
home_team_id, away_team_id	FK to teams (restrict)	Must differ
match_date, kickoff_time	DATE, TIME null	
venue, referee	VARCHAR, null	
status	VARCHAR(20)	scheduled live completed postponed cancelled
home_score, away_score	TINYINT, null	Required for live/completed; cleared otherwise
matchday, attendance	INT, null	
featured	BOOLEAN	Highlight on homepage
report	TEXT, null	Short summary

Table 3.11: match_events

Field	Type	Description / constraint
id	BIGINT, PK	
fixture_id	FK to fixtures (cascade)	
team_id	FK to teams (restrict)	Must be a team in the fixture
player_id	FK to players, null	Scorer / booked player / player off
related_player_id	FK to players, null	Assist provider / player on

Field	Type	Description / constraint
type	VARCHAR(20)	goal penalty_scored own_goal assist yellow_card red_card substitution penalty_missed
minute, additional_minute	TINYINT	e.g. 45 + 2
description	VARCHAR, null	Note

Table 3.12: fixture_players (line-ups)

Field	Type	Description / constraint
id	BIGINT, PK	
fixture_id	FK (cascade)	Unique with player_id
player_id, team_id	FK	
is_starting, is_captain	BOOLEAN	Starting XI / captain
position, shirt_number, minutes_played	VARCHAR / TINYINT, null	

Table 3.13: Content and communication tables (summary)

Table	Key fields
news_posts	title, slug, excerpt, content (sanitised HTML), featured_image, category, author_id (FK to users), fixture_id (FK to fixtures), published_at, is_published, is_featured, deleted_at
gallery_items	title, image, category, fixture_id, team_id, description, is_featured, sort_order
videos	title, slug, thumbnail, video_url, platform, category, description, published_at, is_featured
staff	name, slug, photo, role, type (management coaching), team_id, bio, sort_order, is_active, deleted_at
trial_applications	full_name, email, phone, matric_number, department, level, preferred_position, dominant_foot, height, previous_team, playing_experience, status, admin_notes
contact_messages	name, email, phone, subject, message, status (new read replied archived), admin_notes
users	name, email, password (hashed), role (admin user) — Laravel Breeze authentication

3.10 Algorithm Design

3.10.1 League table computation

The league table is computed on demand by the *LeagueTableService* (Figure 3.5). Every team drawn in the competition receives a row, so that the table is complete even before the first result. Completed fixtures are processed in date order; for each, both teams' played, goals for and goals against are updated, the result adds a win, draw or loss and 3, 1 or 0 points, and the result is appended to the team's form. Rows are then

sorted by points, goal difference, goals for and name, and positions are assigned. Because the table is a function of the results, entering or correcting any result is reflected immediately and consistently.

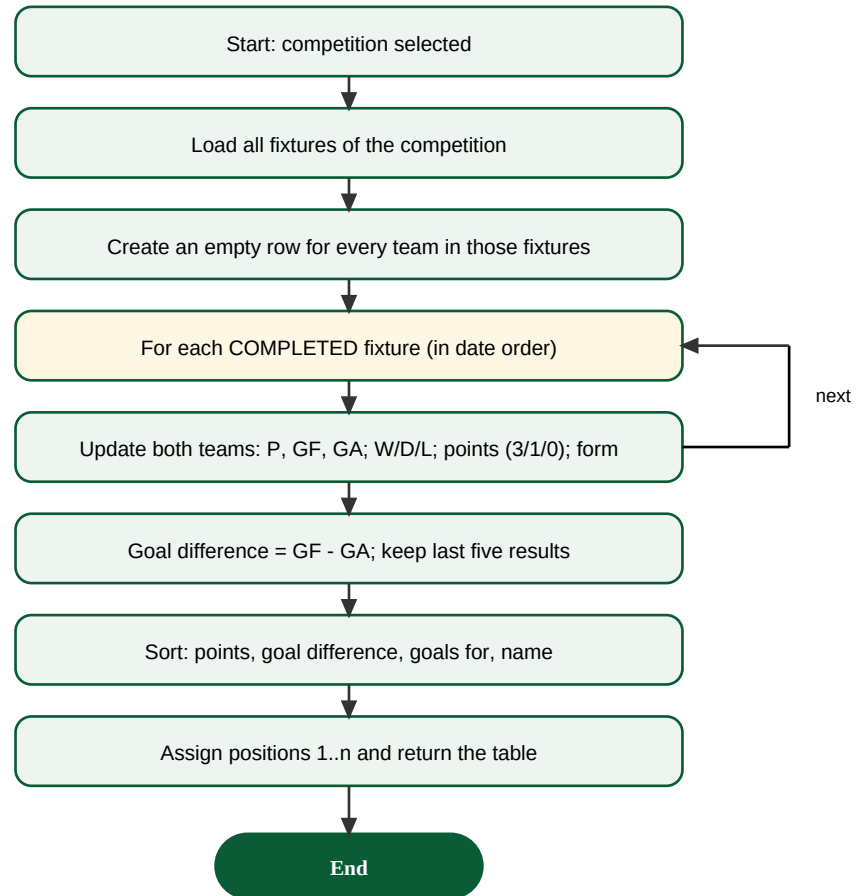


Figure 3.5: Flowchart of the league table algorithm

3.10.2 Player statistics

The *PlayerStatisticsService* derives each player's statistics from live and completed matches only:

- **Appearances** — distinct fixtures in which the player is in a line-up, is involved in an event, or comes on as a substitute.
- **Goals** — goal and penalty-scored events for the player. **Assists** — assist events plus goals on which the player is recorded as the assisting player.
- **Yellow and red cards** — card events for the player.
- **Clean sheets** (goalkeepers) — completed fixtures the goalkeeper started in which the opponent scored zero.

Team statistics and portal-wide totals are computed with single aggregate SQL queries (COUNT and conditional SUM) rather than by loading records into memory.

3.11 Input and Output Design

Inputs are captured through validated forms: administrative forms for every entity (with dropdowns restricted to valid options, date and time pickers, colour pickers, file inputs limited to JPG, PNG and WebP images), the result and event forms on the match page, the public trial application and contact forms (with a hidden honeypot field against spam bots), and CSV files for bulk import. Validation errors are displayed beside the relevant field with an icon and message, not colour alone.

Outputs are the public pages — homepage, fixtures and results, match centre, league table with form guide and top scorers, teams, squad, player profiles, news, gallery, videos and staff pages — and the administrative dashboard with counts, upcoming fixtures, recent results and inbox alerts. Optional email notifications are sent for new contact messages.

3.12 Security Design

Table 3.14: Security measures

Threat	Countermeasure implemented
Unauthorised access to administration	Authentication (Breeze) plus an <i>admin</i> middleware that checks the user's role on every admin route; form requests re-check authorisation; the role cannot be mass-assigned.
SQL injection	All database access through Eloquent / query builder with bound parameters.
Cross-site scripting (XSS)	Blade escapes all output by default; rich text is cleaned by an allow-list HTML sanitiser on save and on display; javascript: links, scripts, event handlers and inline styles are removed.
Cross-site request forgery	CSRF token on every state-changing form, verified by Laravel middleware.
Tampered form data	Server-side validation of every relationship: a match event's team must be in the fixture and its players must belong to that team; IDs from hidden inputs are never trusted.
Malicious file upload	Only images (JPG, PNG, WebP) within size limits are accepted; files are re-encoded to WebP under random names, which also strips embedded metadata such as GPS location.
Spam and abuse of public forms	Honeypot field and rate limiting (5 submissions per minute).
Password compromise	Passwords stored with bcrypt hashing; no default admin password is created outside the local environment.
Privacy of student data	Matriculation numbers, state of origin and applicants' contact details are shown only in the admin panel and hidden from model serialisation.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter describes the implementation environment, the structure of the implemented system and its modules, the testing carried out and the results obtained. Screenshots in this chapter were captured from the running system populated with the generated sample season described in Section 4.4.

4.2 Implementation Environment

Table 4.1: Development and runtime environment

Component	Specification
Operating system	Microsoft Windows 11
Local server stack	XAMPP (Apache, MariaDB 10.4.32) and PHP's built-in server
Language / framework	PHP 8.2.12; Laravel Framework 12.69
Database	MariaDB 10.4.32 (MySQL-compatible); SQLite in-memory for automated tests
Front-end tooling	Node.js 22, Vite 7, Tailwind CSS 3.4, Alpine.js 3, Trix 2
Libraries	Laravel Breeze 2, Symfony HtmlSanitizer 7.4, PHP GD and EXIF extensions
Testing	Pest 3 on PHPUnit; headless Microsoft Edge (Chrome DevTools Protocol) for responsive checks
Editor	Visual Studio Code
Minimum client	Any modern browser (Chrome, Edge, Firefox, Safari) on phone, tablet or desktop

4.3 System Structure and Modules

The implemented system comprises 14 Eloquent models, 38 controller classes (public and admin), 17 database migrations, 115 Blade view files, 4 service classes and 121 named routes, of which 71 belong to the administration panel. Application PHP code amounts to about 5,400 lines and view templates to about 5,200 lines.

Table 4.2: Main modules

Module	Key components	Function
Public website	HomeController, TeamController, PlayerController, FixtureController, LeagueTableController, NewsController, GalleryController, VideoController, StaffController	Presents all competition information and content.

Module	Key components	Function
Standings and statistics	LeagueTableService, TeamStatisticsService, PlayerStatisticsService	Derives tables and statistics from results and events.
Administration	Admin* controllers, Form Requests, admin Blade components	CRUD for all entities; result, event and line-up entry; inboxes.
Authentication and access control	Laravel Breeze, EnsureUserIsAdmin middleware, users.role	Login and restriction of the admin panel.
Forms and communication	TrialApplicationController, ContactController, ContactMessageReceived mailable	Trial applications and contact messages.
Content editing	Trix editor, RichText sanitiser	Safe rich-text news articles.
Media	ImageOptimizer, ManagesUploads	Resizing, orientation, WebP conversion and cleanup of uploads.
Data loading	FootballImporter, football:import command, SampleDataSeeder, SampleGraphics	CSV import of real data; generation of a sample season and graphics.

4.4 Test Data

Verified squad lists were not available during development (Section 1.6). To exercise every feature realistically, a deterministic *SampleDataSeeder* generates a complete season for the seven real teams: 126 fictional players (18 per team) with realistic Nigerian names, departments and levels; 22 fictional management and coaching staff; a full double round-robin league of 42 fixtures, of which matches already due are played with consistent results, goals, assists, cards, substitutions and line-ups; match reports; and gallery items. Illustrations (team crests, player shirt cards and staff silhouettes) are drawn by the system rather than using photographs of people. The seeder is disabled by default outside development, and real data is loaded with the *football:import* command (Appendix B).

4.5 Implementation Screenshots

4.5.1 Public website

Figures 4.1 to 4.9 show the public website. The homepage presents the club identity, featured players, latest news, the next fixture with a live kick-off countdown, recent results and statistics computed from the database.

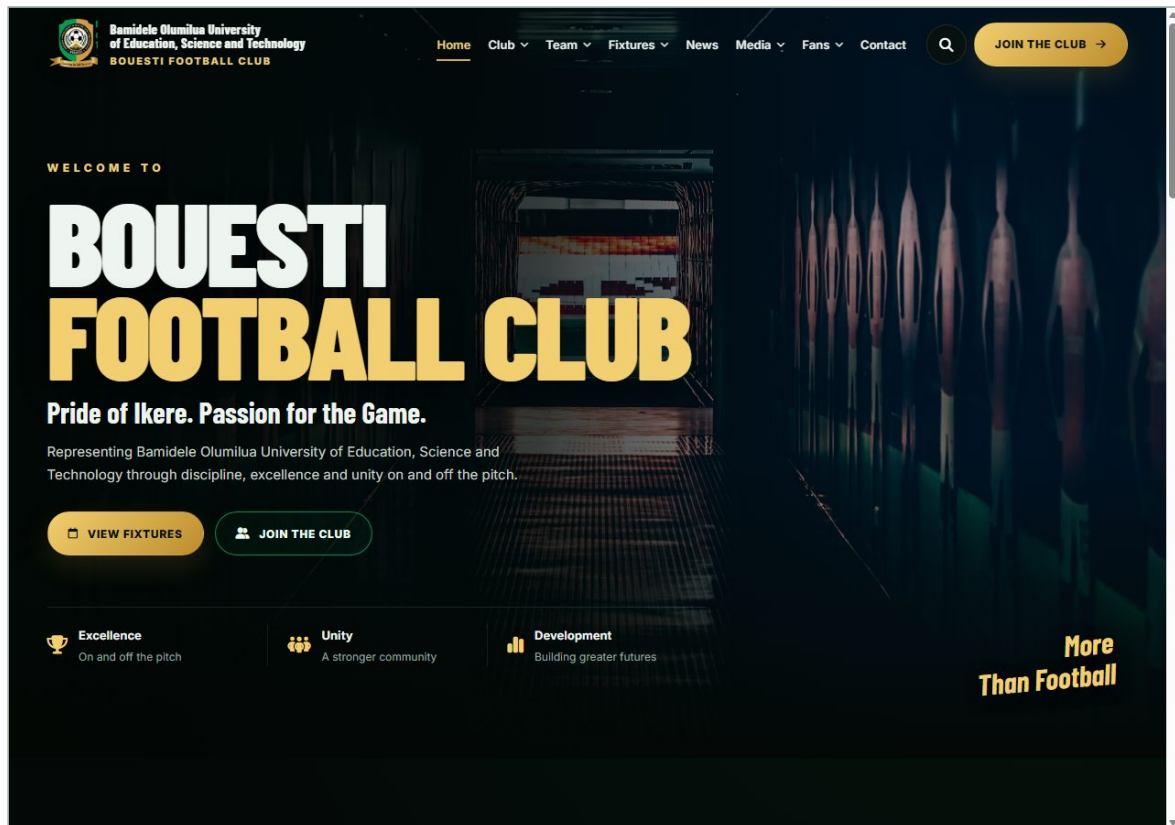


Figure 4.1: Homepage

2026/2027

BOUESTI FOOTBALL LEAGUE TABLE

#	TEAM	P	W	D	L	GF	GA	GD	FORM	PTS
1	CSC Elites	5	3	2	0	7	3	+4	W W D D W	11
2	Amapro FC	5	3	1	1	10	4	+6	L W W D W	10
3	AMCOMS	5	2	1	2	6	7	-1	L W D W L	7
4	Sovereignty FC	5	1	3	1	6	5	+1	D L D D W	6
5	Elite FC	5	1	3	1	4	4	0	W D D D L	6
6	Engines Boys	5	1	2	2	2	5	-3	L D D L W	5
7	Young Boys FC	6	0	2	4	2	9	-7	L D L L L	2

Sorted by points, then goal difference, then goals scored. Updated automatically after every result.

Figure 4.2: League table with form guide, computed from completed fixtures

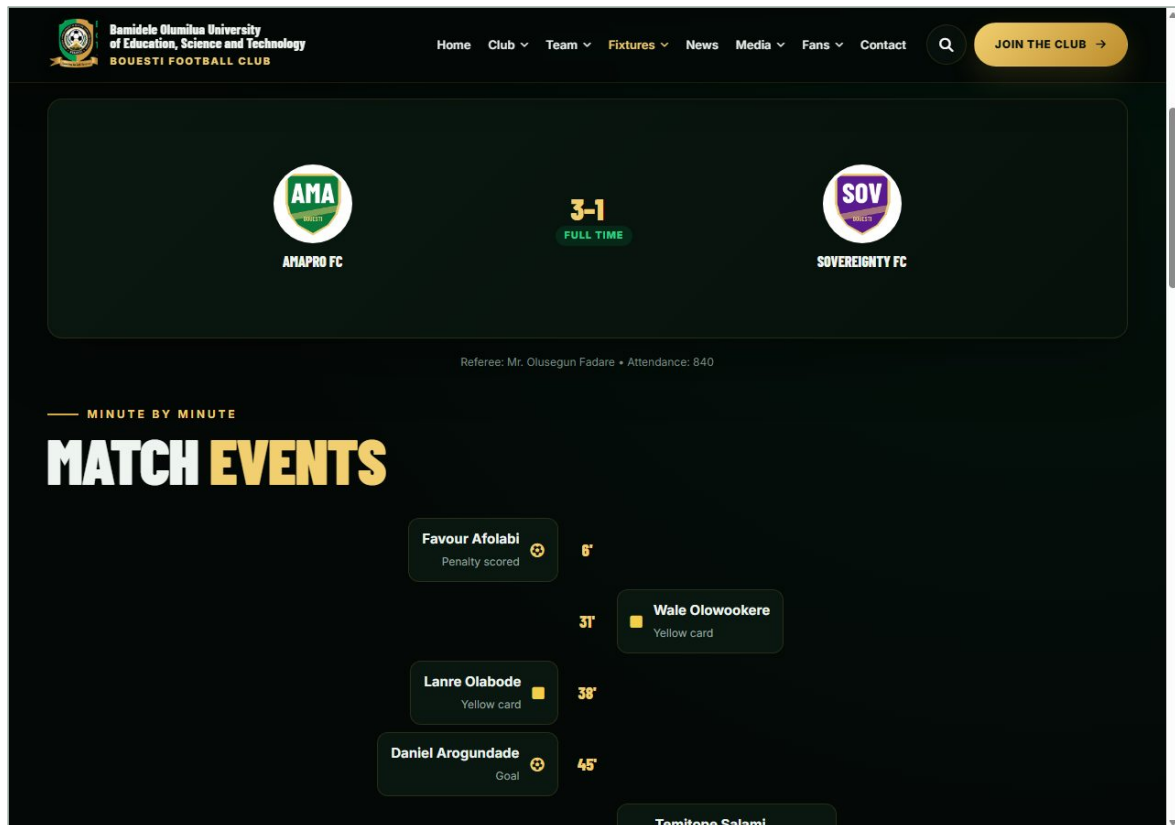


Figure 4.3: Match centre with score and minute-by-minute events

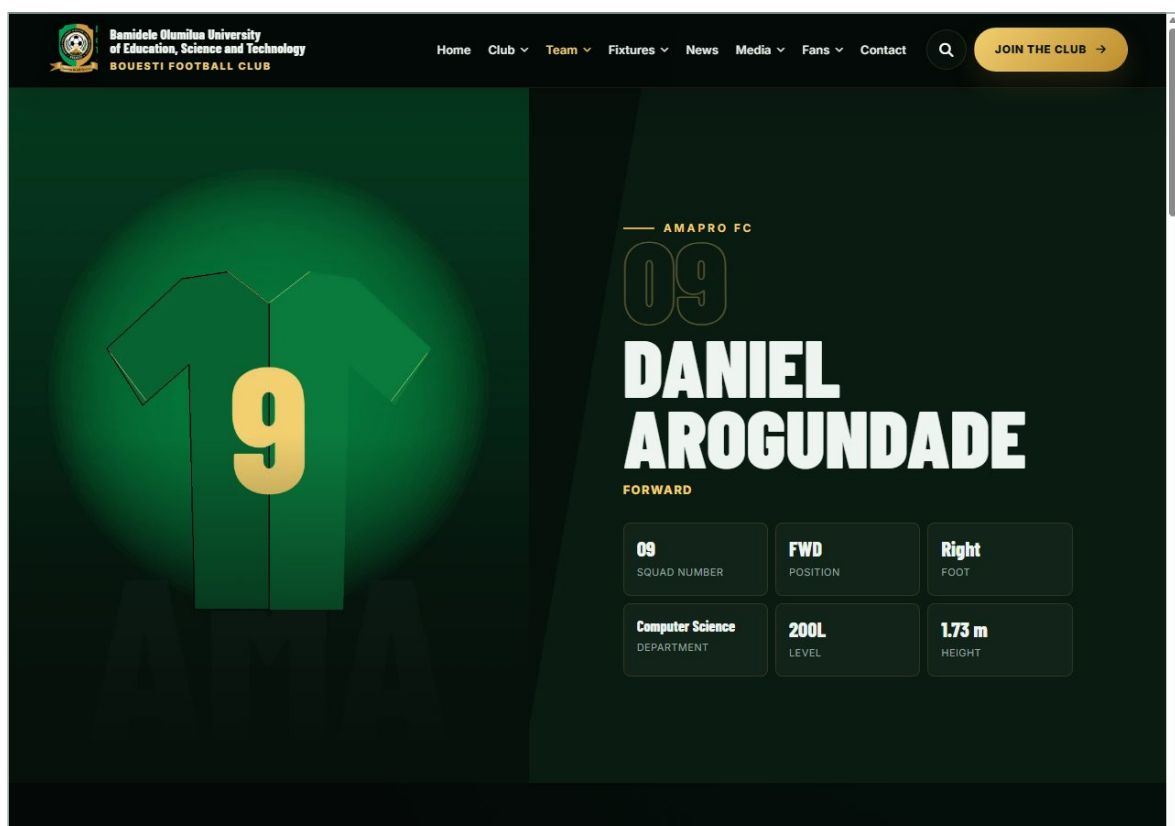


Figure 4.4: Player profile (public details only; matriculation number is never shown)

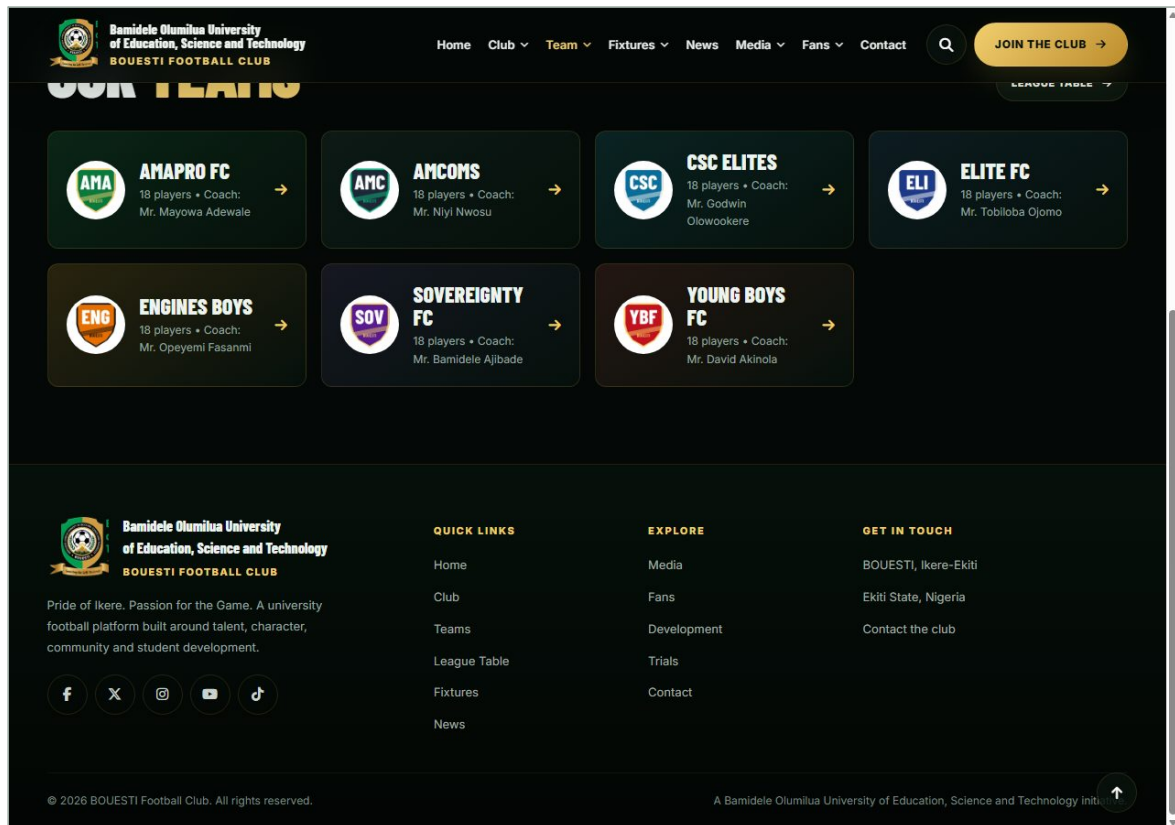


Figure 4.5: Teams directory with generated crests



Figure 4.6: Squad page with team and position filters

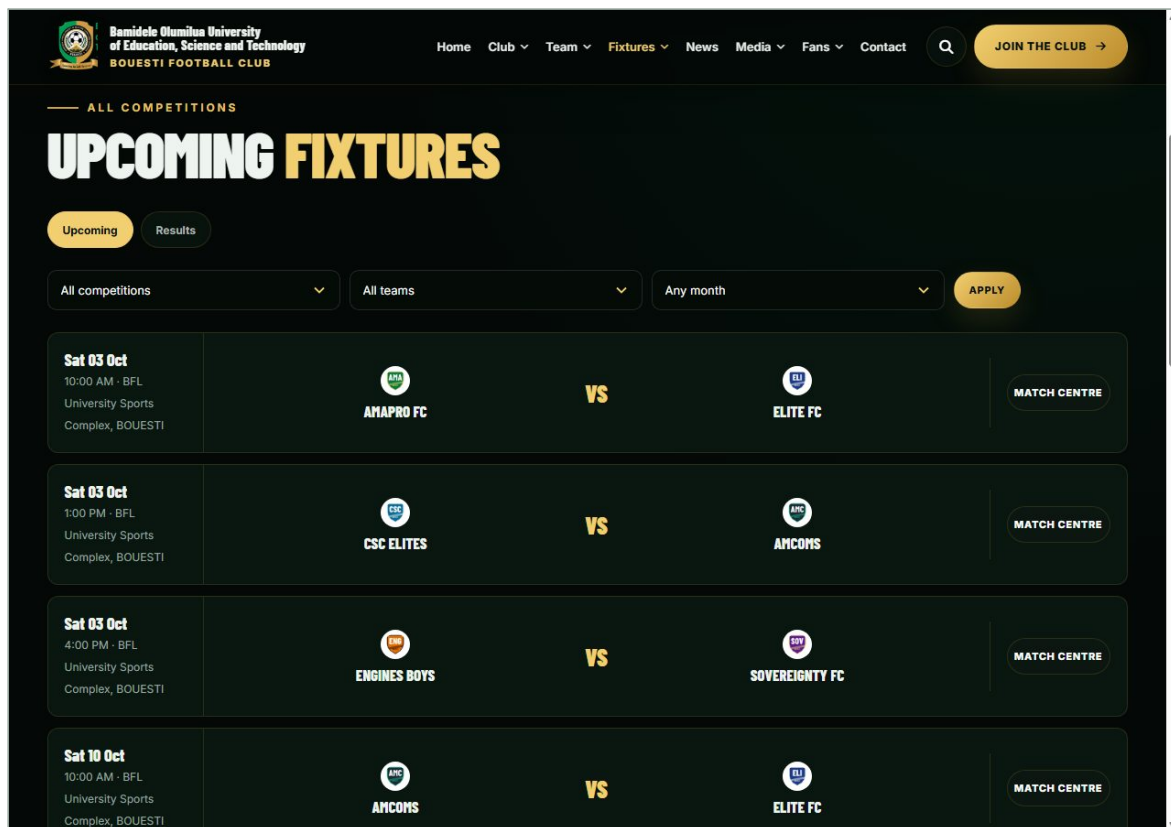


Figure 4.7: Fixtures with competition, team and month filters

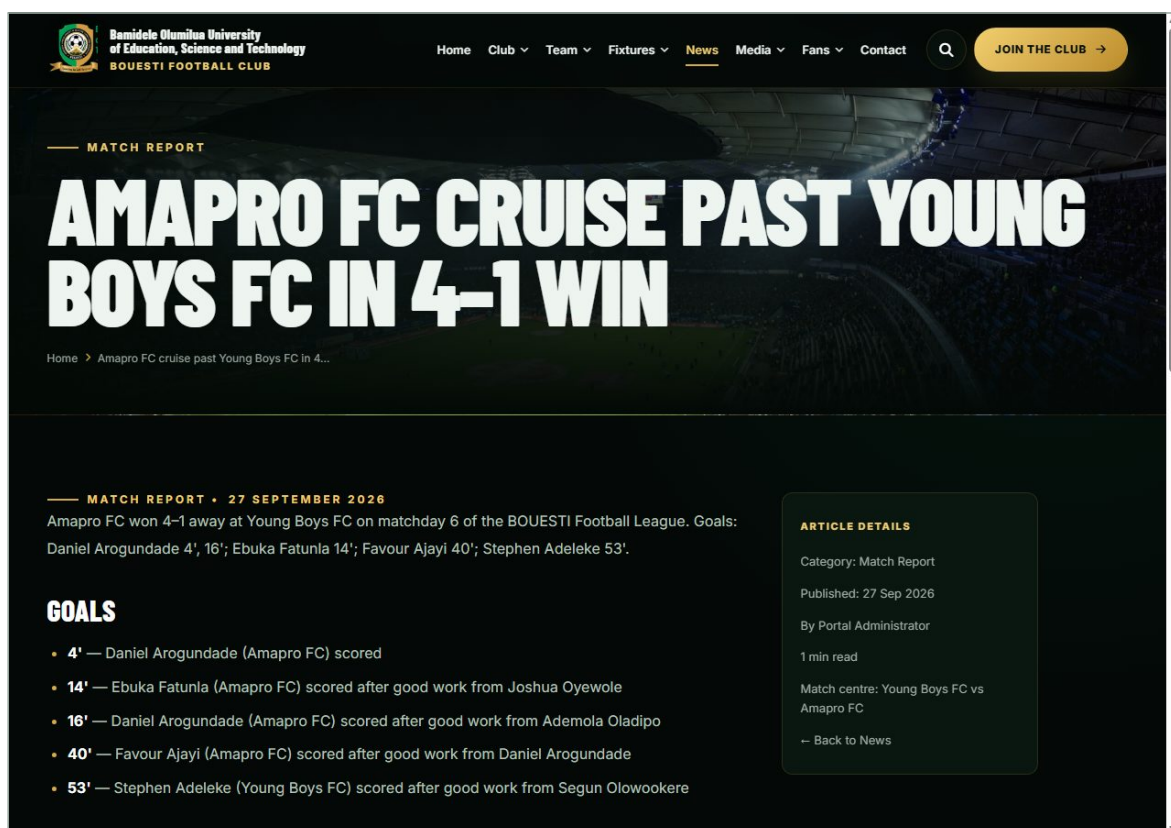


Figure 4.8: Match report article

Bamidele Olumilua University of Education, Science and Technology
BOUESTI FOOTBALL CLUB

Home Club Team Fixtures News Media Fans Contact

JOIN THE CLUB →

JOIN THE CLUB

FOOTBALL TRIALS

A responsive registration experience ready to connect to a future Laravel backend.

Home > Football Trials

JOIN BOUESTI FC

YOUR NEXT STEP

Want to play for a BOUESTI team? Register your interest below. Applications are reviewed by the football coordinators, who will contact you with trial dates.

ELIGIBILITY

Open to registered BOUESTI students. Bring your student ID to trials. Your matric number and contact details are only seen by the football administrators.

Full Name *

Your full name

Email Address *

name@example.com

Phone Number *

+234

Matric Number *

Student matric number

Department

Your department

Level *

Select level

Preferred Position *

Dominant Foot

Figure 4.9: Online trial application form

4.5.2 Mobile views

The same pages adapt to phone screens: navigation moves into an accessible slide-in menu, grids collapse to a single column and the league table keeps the position and club columns fixed while statistics scroll (Figure 4.10).

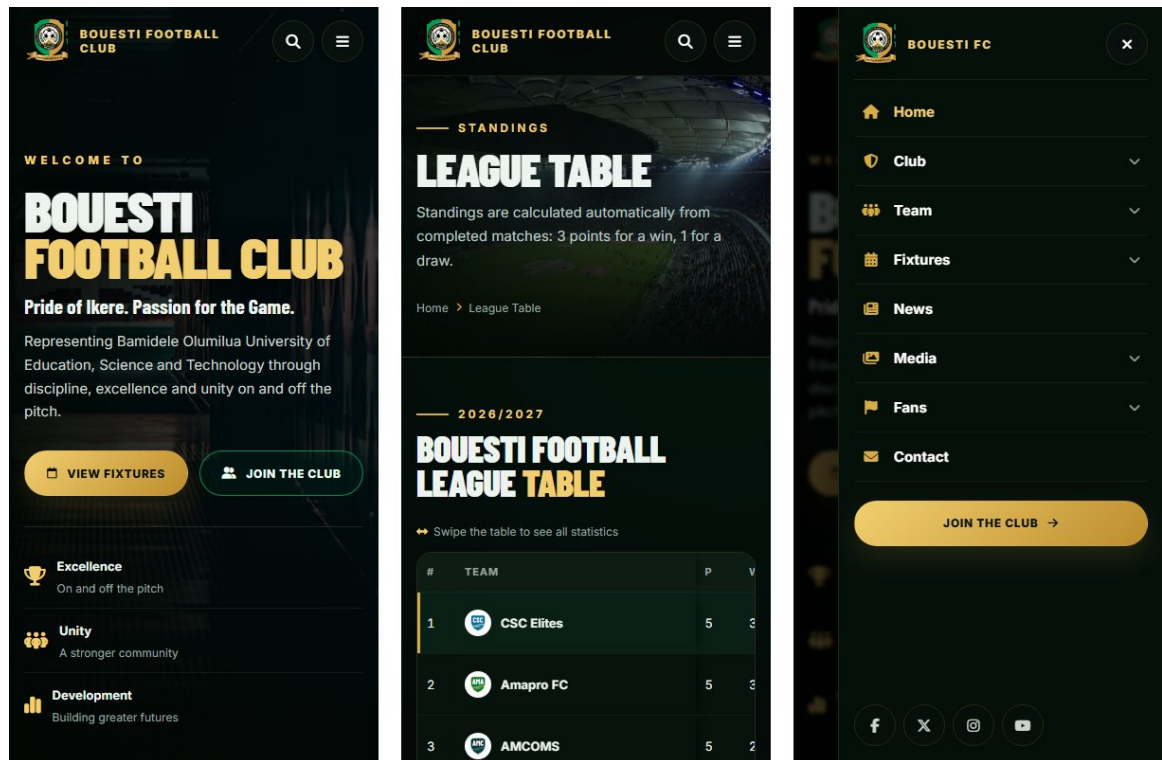


Figure 4.10: Mobile views — homepage, league table and navigation menu

4.5.3 Administration panel

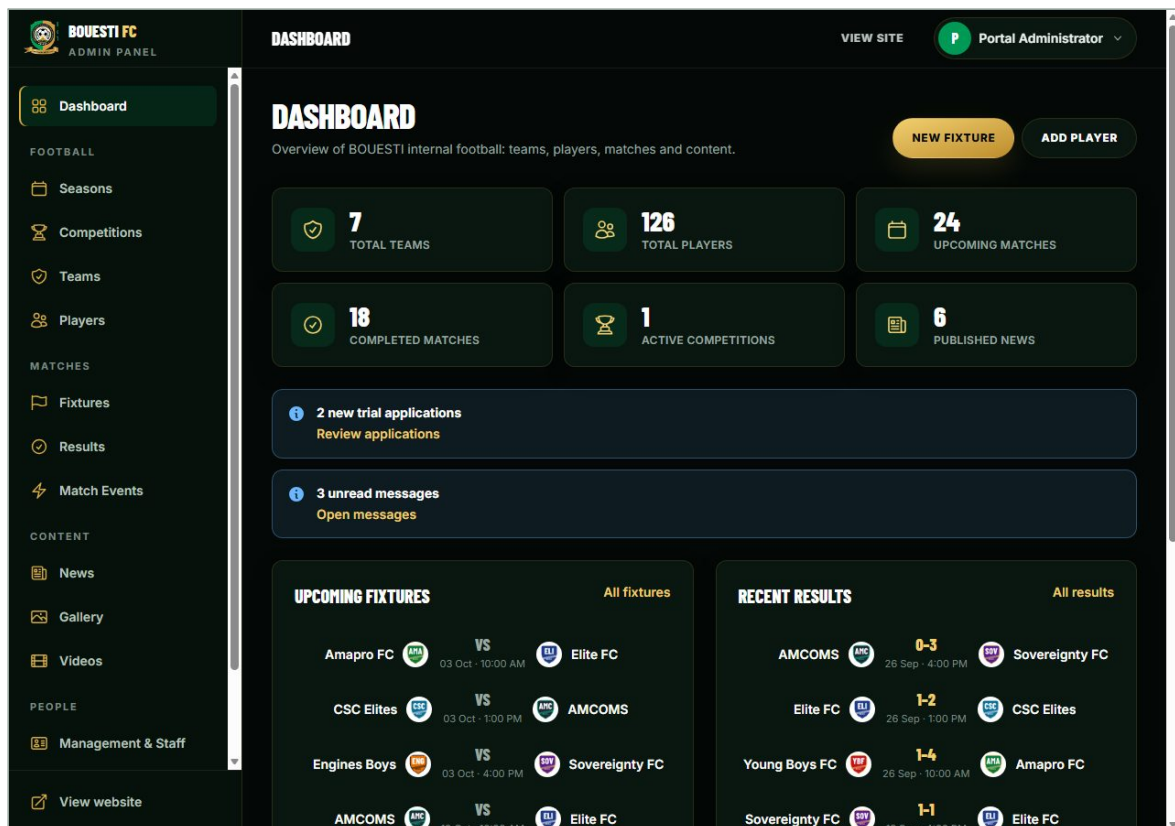


Figure 4.11: Administration dashboard

BOUESTI FC
ADMIN PANEL

Dashboard
FOOTBALL
Seasons
Competitions
Teams
Players
MATCHES
Fixtures
Results
Match Events
CONTENT
News
Gallery
Videos
PEOPLE
Management & Staff
View website

AMAPRO FC VS SOVEREIGNTY FC

VIEW SITE Portal Administrator

Back

AMAPRO FC VS SOVEREIGNTY FC

BOUESTI Football League · Matchday 3 · Saturday 05 September 2026 · 10:00 AM · University Sports Complex, BOUESTI

EDIT FIXTURE PUBLIC PAGE

AMA

Amapro FC

HOME

3-1

SOV

Sovereignty FC

AWAY

RESULT

Saving a completed result updates standings, team and player statistics immediately.

Status

Full time

Amapro FC

3

Sovereignty FC

1

Referee

Mr. Olusegun Fadar

Attendance

840

Short match summary

MATCH EVENTS

Goals, assists, cards and substitutions feed player statistics.

6'

Penalty scored — Favour Afolabi

Amapro FC

31'

Yellow card — Wale Olowookere

Sovereignty FC

38'

Yellow card — Lanre Olabode

Amapro FC

45'

Goal — Daniel Arogundade

Amapro FC

55'

Substitution — Temitope Salami (On: Sola Suleiman)

Sovereignty FC

Substitution — Ikenna Lawal (On: Adebayo

Figure 4.12: Match management — result entry and match events

BOUESTI FC
ADMIN PANEL

Dashboard
FOOTBALL
Seasons
Competitions
Teams
Players
MATCHES
Fixtures
Results
Match Events
CONTENT
News
Gallery
Videos
PEOPLE
Management & Staff
View website

PLAYERS

VIEW SITE Portal Administrator

PLAYERS

Student players across all BOUESTI teams. Matric numbers are visible to admins only.

ADD PLAYER

Q Name or matric number

All teams

All positions

FILTER

PLAYER	#	TEAM	POSITION	MATRIC NO.	STATUS			
1 Gbenga Adebayo	1	Amapro FC	Goalkeeper	80U/24/1443	ACTIVE			
16 Tobiloba Adewale	16	Amapro FC	Goalkeeper	80U/24/6299	ACTIVE			
2 Precious Lawal	2	Amapro FC	Defender	80U/25/7543	ACTIVE			
3 Ebuka Fatunla	3	Amapro FC	Defender	80U/25/8189	ACTIVE			
4 Favour Adegoke	4	Amapro FC	Defender	80U/24/3978	ACTIVE			
5 Godwin Eze	5	Amapro FC	Defender	80U/24/4512	ACTIVE			
12 Kunle Adeleke	12	Amapro FC	Defender	80U/23/5724	ACTIVE			
15 Emmanuel Anyanwu	15	Amapro FC	Defender	80U/23/9837	ACTIVE			
6 Oreoluwa Oluwadare	6	Amapro FC	Midfielder	80U/23/8647	ACTIVE			

Figure 4.13: Player management with search and filters (admin-only matriculation numbers)

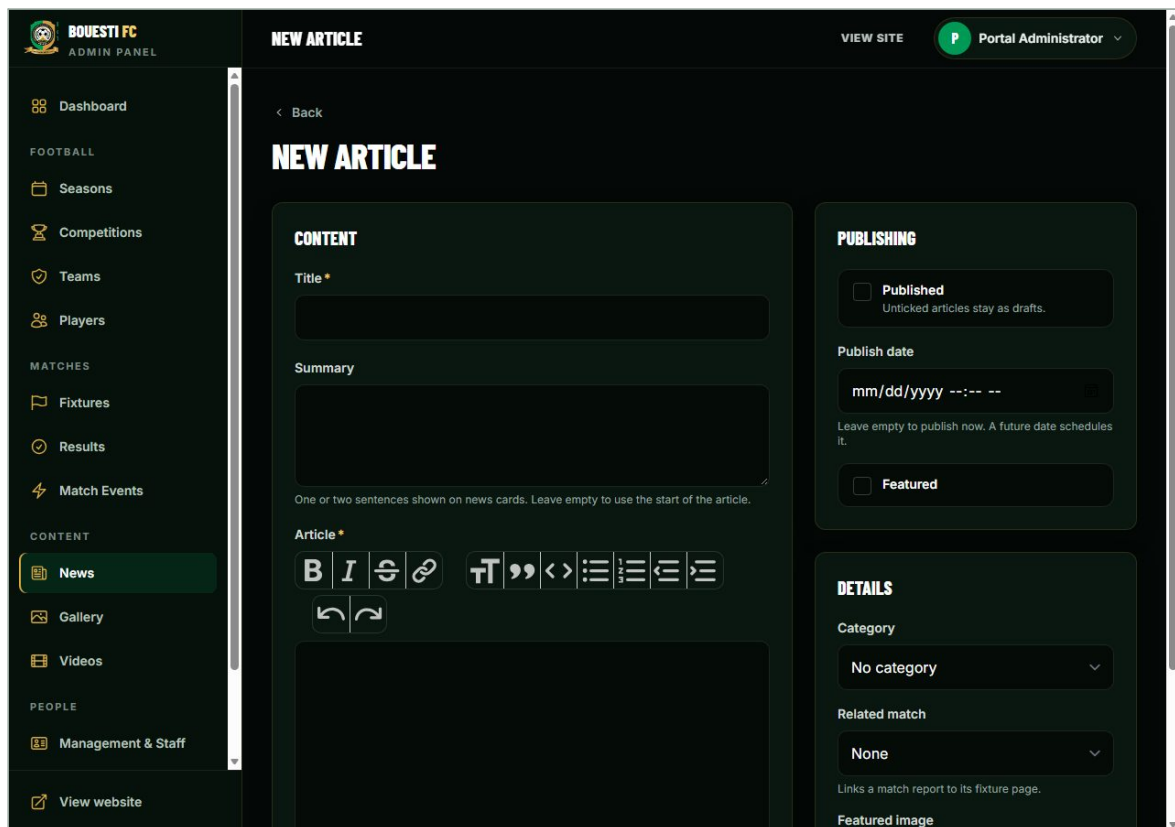


Figure 4.14: News editor with rich-text toolbar

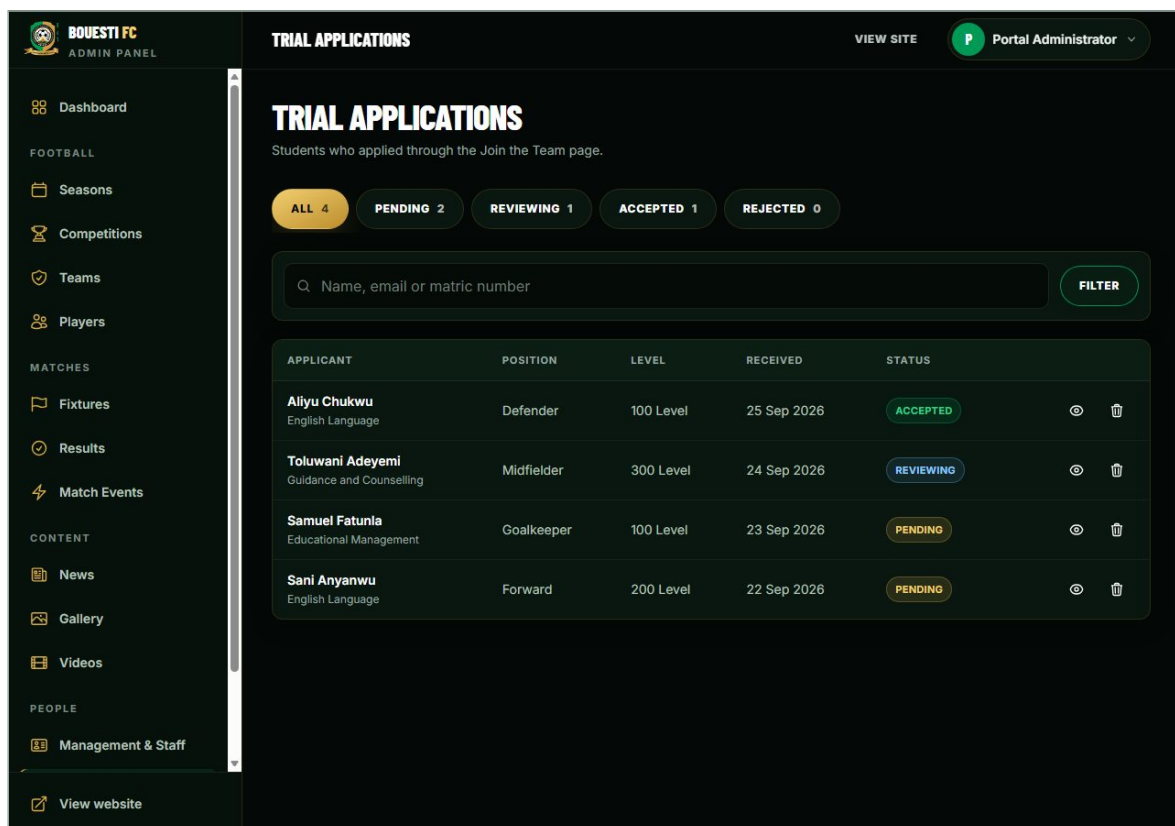


Figure 4.15: Trial applications inbox

4.6 System Testing

4.6.1 Test strategy

Testing combined automated feature tests, automated responsive-layout audits and manual exploratory testing. Automated tests run with Pest against a fresh in-memory SQLite database for every test, so each test starts from a known state and does not affect development data. Feature tests drive the application through HTTP requests exactly as a browser would — submitting forms, following redirects and inspecting rendered pages — and therefore test routes, middleware, validation, controllers, services, models and views together.

4.6.2 Test cases and results

Table 4.3: Summary of test cases and results

ID	Test case	Expected result	Result
TC01	Seven internal teams seeded with correct slugs	amapro-fc ... amcoms created; one current season	Pass
TC02	Guest opens admin panel; ordinary user opens or posts to admin	Guest redirected to login; user receives 403; no data created	Pass
TC03	End-to-end workflow: add players to Amapro FC and Elite FC, create fixture, enter 2–1 result, record goal with assist, publish match report	Fixture listed publicly; table shows Amapro FC 3 pts first; scorer has 1 goal and 1 appearance; assister 1 assist; result on homepage; report published	Pass
TC04	Create fixture with the same home and away team	Rejected with message; nothing saved	Pass
TC05	Complete a match without the away score; postpone a match with scores	Validation error; scores cleared for postponed match	Pass
TC06	Duplicate shirt number in same team / different team	Rejected in same team; allowed across teams	Pass
TC07	Record event with a player from the other team, or a team not in the fixture (tampered form)	Both rejected; no events stored	Pass
TC08	Mark a second season as current	Only the new season is current	Pass
TC09	Delete a team that has played	Team soft-deleted; historical match still shows its name	Pass
TC10	Delete a competition that has fixtures	Refused with explanatory message	Pass
TC11	Line-up including a player from another team	Rejected; valid line-up saved with starter and captain	Pass
TC12	Upload team logo; replace it; upload a PHP file as logo	Stored on public disk; old file deleted; PHP file rejected	Pass
TC13	Submit trial application (valid, invalid, honeypot filled)	Stored as pending; errors shown; spam rejected	Pass
TC14	Visit all 25 public pages with an empty database	All return HTTP 200 with empty-state messages	Pass
TC15	Open legacy static URLs	Redirect to dynamic pages	Pass

ID	Test case	Expected result	Result
TC16	Open an unpublished article publicly	HTTP 404	Pass
TC17	Open every admin screen	All render; opening an application marks it “reviewing”	Pass
TC18	Contact form with and without notification address	Message stored; email sent with reply-to only when configured	Pass
TC19	Contact form validation, honeypot, confirmation; admin inbox workflow	Errors shown; message marked read / replied; non-admins refused	Pass
TC20	Upload a 4000×3000 photo; upload a 300×200 image	Stored as 2000×1500 WebP; small image not enlarged	Pass
TC21	Rich text containing script, onerror, javascript: link, iframe, style	Formatting kept; all dangerous content removed	Pass
TC22	Save article from editor; save empty editor; view legacy plain-text article	Stored clean and rendered; empty rejected; legacy rendered safely	Pass
TC23	CSV import: full import; re-import; invalid rows; dry run; missing photo	Data and statistics created; no duplicates; whole import rolled back with line-numbered errors; dry run saves nothing; missing photo is a warning	Pass
TC24	Sample season generation	126 players, 22 staff, 42 fixtures; every score equals its goal events; 11 starters and 5 substitutes per team; balanced standings	Pass

In total, **91 automated tests with 901 assertions** were executed; all passed. These comprise the 42 project-specific tests grouped in Table 4.3 (several table rows cover multiple tests) together with the authentication, registration, password and profile tests supplied with Laravel Breeze, which confirm that existing account functionality was preserved.

4.6.3 Responsive and usability testing

An automated audit loaded every public page in headless Microsoft Edge with true mobile emulation at ten widths — 320, 360, 375, 390, 414, 430, 768, 1024, 1280 and 1440 pixels (240 page loads) — and checked for horizontal scrolling, elements overflowing the viewport, controls smaller than touch-target guidelines, text below 11 pixels and JavaScript errors. After fixes made during development, no page produced horizontal scrolling, overflow or console errors. Interactive behaviour — mobile menu, search dialog, filters, form validation, accordion, lightbox, sticky table columns, the admin drawer and confirmation dialogs — was also exercised automatically.

4.6.4 Defects found and fixed

Testing revealed and led to the correction of several defects, for example: a blocking font import that delayed page styling on slow connections; hero text touching the screen edge on mobile; a screen-reader-only label causing horizontal scrolling in admin tables; unreadable 10-pixel labels; and an incorrectly ordered score (1–4 instead of 4–1) in

generated match-report headlines. This illustrates the value of combining automated and visual testing.

4.7 Performance Considerations

- Statistics use single aggregate SQL queries; related records are eager-loaded to avoid repeated queries.
- A 4000×3000 phone photograph is stored as a 2000×1500 WebP image, typically a small fraction of the original size.
- The production admin stylesheet is about 13 KB compressed (gzip); the rich-text editor script (about 52 KB compressed) loads only on the news form.
- Images below the fold are lazy-loaded, and phones receive smaller image variants.

4.8 Deployment

The system can be deployed on any host supporting PHP 8.2 and MySQL/MariaDB, including shared hosting. Deployment steps — installing dependencies, configuring the environment, running migrations, linking storage and building assets — are given in Appendix B. Before going live, the administrator password must be set through the environment, sample data disabled, and verified squad data imported.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This project addressed the lack of a central, accurate and accessible source of information for the internal student football competitions of Bamidele Olumilua University of Education, Science and Technology, Ikere-Ekiti. A web-based football league management and information portal was designed and implemented using the Laravel framework, a relational database and a responsive user interface.

The system provides a public website for fixtures, results, automatically computed standings, a match centre, teams, squads, player profiles and statistics, news, a gallery and videos, together with online trial applications and contact messages. A secure administration panel enables officials to manage all data and content, record results, match events and line-ups, and review applications and messages. Supporting features include a sanitised rich-text editor, automatic image optimisation, a CSV import tool for real data and a generator for a realistic sample season. The system was verified with 91 automated tests and responsive audits across ten screen widths.

5.2 Achievement of Objectives

1. A normalised relational database of 17 migrations was designed and implemented for all competition and content entities (Objective 1).
2. A responsive public website presenting all competition information was developed (Objective 2).
3. League standings and player and team statistics are computed automatically from results and events (Objective 3).
4. A role-based administration panel with full management of all data was implemented (Objective 4).
5. Online trial applications and contact messages with administrative inboxes were implemented (Objective 5).
6. Security controls addressing the OWASP Top 10 risks relevant to the system, and privacy rules for student data, were implemented (Objective 6).
7. The system was tested with automated feature tests and responsive audits, all of which passed (Objective 7).

5.3 Conclusion

The study shows that a purpose-built, institution-owned web portal can replace informal handling of internal competition information with a single reliable source of truth. Recording each fact once and deriving standings and statistics from it removes computational errors and disputes, preserves history across seasons, gives players recognition, and makes information instantly available to every student on any device, while role-based access control protects student data. The use of an established MVC framework, a normalised database and automated testing produced a system that is maintainable and can be extended by future students and developers.

5.4 Recommendations

- The university's sports unit should adopt the portal as the official record of internal competitions and assign trained officials to record results and events promptly after each match.
- Verified squad lists, staff details, team colours and logos should be imported before public launch, and players' consent should be obtained before publishing photographs.
- The system should be hosted on the university's infrastructure or a reputable host with HTTPS, regular database backups and a strong administrator password.
- Officials should be trained using the user manual in Appendix A.
- The Department of Computer Science may use the project as a platform for future student projects.

5.5 Suggestions for Further Work

- Real-time live match updates using WebSockets and push notifications to subscribed students.
- Automatic fixture generation (round-robin scheduling) and knockout-bracket visualisation for cup competitions.
- Multiple administrator roles (for example, team managers who can submit their own line-ups) with approval workflows.
- Mobile applications or a Progressive Web App with offline access.
- Advanced analytics and season-to-season historical comparisons.
- Integration with the university's student information system to verify players' registration.

REFERENCES

- Beck, K. (2002). *Test-Driven Development: By Example*. Boston: Addison-Wesley.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- Connolly, T., and Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Harlow: Pearson.
- Elmasri, R., and Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Hoboken, NJ: Pearson.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Krasner, G. E., and Pope, S. T. (1988). A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming*, 1(3), 26–49.
- Laravel. (2025). *Laravel 12.x Documentation*. Retrieved from <https://laravel.com/docs/12.x>
- Marcotte, E. (2010, May 25). Responsive web design. *A List Apart*. Retrieved from <https://alistapart.com/article/responsive-web-design/>
- Nielsen, J. (1993). *Usability Engineering*. Boston: Academic Press.
- OWASP Foundation. (2021). *OWASP Top 10:2021*. Retrieved from <https://owasp.org/Top10/>
- Pest. (2025). *Pest PHP Testing Framework Documentation*. Retrieved from <https://pestphp.com/docs>
- Pressman, R. S., and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). New York: McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Boston: Pearson.
- Symfony. (2025). *The HtmlSanitizer Component*. Retrieved from https://symfony.com/doc/current/html_sanitizer.html
- Tailwind Labs. (2025). *Tailwind CSS Documentation*. Retrieved from <https://tailwindcss.com/docs>
- World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Retrieved from <https://www.w3.org/TR/WCAG21/>

APPENDIX A: USER MANUAL

A.1 Visitors and students. Open the website in any browser. Use the menu (the menu button on phones) to reach Teams, Squad, Fixtures, League Table, News, Media and Fans pages; use the search button to find any page. On the Fixtures page, switch between Upcoming and Results and filter by competition, team or month; select *Match Centre* for a match's events and line-ups. To apply for trials, open *Join the Club*, complete the form and submit; to contact the club, use the Contact page.

A.2 Logging in as an administrator. Go to */login*, enter the administrator email and password, and you are taken to */admin/dashboard*. The sidebar groups all functions under Football, Matches, Content and People; on phones, open it with the menu button.

A.3 Setting up a season.

1. Seasons > New season: enter e.g. 2026/2027 and tick *Current season*.
2. Competitions > New competition: choose the season and type (League for a table).
3. Teams: edit each team to add the short name, colours, logo, coach and description.
4. Players > Add player: choose the team and position, enter the student's details and photo; use *Save and add another* for quick entry. Matriculation number and state of origin remain private.

A.4 Scheduling and recording matches.

1. Fixtures > New fixture: choose competition, home and away teams, date, kick-off time, venue and matchday.
2. After the match, open the fixture. In *Result*, choose *Full time*, enter both scores and save — the league table updates immediately.
3. In *Match events*, choose the event type and team, then the player (only that team's players are listed); add the assisting player for goals or the player coming on for substitutions; enter the minute and save.
4. In *Line-ups*, tick each team's match-day squad, the starters (maximum 11) and the captain, then save.
5. Select *Write match report* to publish a news article linked to the match, and *Add photos* to upload match photos.

A.5 Content and inboxes. News: write with the toolbar, choose a category, tick *Published* (or set a future date to schedule). Gallery: upload up to 20 photos at once. Videos: paste a YouTube link. Management and Staff: add people with type Management or Coaching. Trial Applications and Messages: open an item to read it (it is marked as read), change its status and add private notes.

A.6 Deleting. Every delete asks for confirmation. Teams, players, news and staff are archived rather than destroyed, so past results remain correct. Competitions and seasons that have fixtures cannot be deleted; mark them inactive instead.

APPENDIX B: INSTALLATION AND DEPLOYMENT GUIDE

Requirements: PHP 8.2+ with GD, EXIF and PDO MySQL extensions; Composer 2; Node.js 18+; MySQL 8 or MariaDB 10.4+.

```
# 1. Install dependencies
composer install --no-dev --optimize-autoloader
npm install && npm run build

# 2. Configure the environment
cp .env.example .env
php artisan key:generate
#   edit .env: APP_URL, DB_DATABASE, DB_USERNAME, DB_PASSWORD,
#   ADMIN_EMAIL, ADMIN_PASSWORD, MAIL_* settings, CONTACT_NOTIFY_EMAIL,
#   APP_TIMEZONE=Africa/Lagos, SEED_SAMPLE_DATA=false (production)

# 3. Database, storage and caches
php artisan migrate --seed          # season, league, seven teams, administrator
php artisan storage:link
php artisan optimize

# 4. Load verified data (fill database/data/import/*.csv, photos in photos/)
php artisan football:import --dry-run
php artisan football:import

# Development only: complete sample season with fictional people
#   SEED_SAMPLE_DATA=true then php artisan migrate:fresh --seed

# Run the automated tests
php artisan test
```

For shared hosting, point the domain's document root to the project's *public* folder. Enable HTTPS and schedule regular database backups.

APPENDIX C: SELECTED SOURCE CODE

C.1 League table computation (app/Services/LeagueTableService.php)

```
<?php

namespace App\Services;

use App\Models\Competition;
use App\Models\Fixture;
use App\Models\Season;
use App\Models\Team;
use Illuminate\Support\Collection;

/**
 * Builds standings from COMPLETED fixtures – points are never stored.
 * Win 3, draw 1, loss 0. Sorted by points, goal difference, goals for, then name.
 */
class LeagueTableService
{
    public const WIN = 3;
    public const DRAW = 1;

    /**
     * @return Collection<int, object{position:int, team:Team, played:int, won:int,
     *     drawn:int, lost:int,
     *     goals_for:int, goals_against:int,
     *     goal_difference:int, points:int, form:array}>
     */
    public function generate(Competition $competition, ?Season $season = null): Collection
    {
        $fixtures = Fixture::query()
            ->where('competition_id', $competition->id)
            ->when($season, fn ($q) => $q->where('season_id', $season->id))
            ->get(['id', 'home_team_id', 'away_team_id', 'status', 'home_score',
                'away_score', 'match_date', 'kickoff_time']);

        // Every team drawn into the competition appears, even before its first result.
        $teamIds = $fixtures->pluck('home_team_id')->merge($fixtures->pluck('away_team_id'))
            ->unique();
        if ($teamIds->isEmpty()) {
            return collect();
        }

        $rows = Team::withTrashed()->whereIn('id', $teamIds)->get()->mapWithKeys(fn (Team
            $team) => [
                $team->id => (object) [
                    'position' => 0, 'team' => $team, 'played' => 0, 'won' => 0, 'drawn' => 0,
                    'lost' => 0,
                    'goals_for' => 0, 'goals_against' => 0, 'goal_difference' => 0, 'points' =>
                        0, 'form' => [],
                ],
            ]);

        $completed = $fixtures
            ->filter(fn (Fixture $f) => $f->status === 'completed' && $f->home_score !==
                null && $f->away_score !== null)
            ->sortBy(fn (Fixture $f) => $f->match_date->format('Y-m-d') . ' ' .
                $f->kickoff_time . sprintf('%08d', $f->id));

        foreach ($completed as $fixture) {
            $this->record($rows[$fixture->home_team_id], $fixture->home_score,
                $fixture->away_score);
            $this->record($rows[$fixture->away_team_id], $fixture->away_score,
                $fixture->home_score);
        }

        return $rows->values()
            ->each(function ($row) {
                $row->goal_difference = $row->goals_for - $row->goals_against;
            });
    }
}
```

```

        $row->form = array_slice($row->form, -5);
    })
    ->sort(fn ($a, $b) => [$b->points, $b->goal_difference, $b->goals_for,
        $a->team->name]
        <=> [$a->points, $a->goal_difference, $a->goals_for, $b->team->name])
    ->values()
    ->each(fn ($row, $i) => $row->position = $i + 1);
}

/** A team's current position in a competition, or null if it is not in the table. */
public function positionOf(Team $team, Competition $competition, ?Season $season =
    null): ?object
{
    return $this->generate($competition, $season)->first(fn ($row) => $row->team->id
        === $team->id);
}

private function record(object $row, int $for, int $against): void
{
    $row->played++;
    $row->goals_for += $for;
    $row->goals_against += $against;

    if ($for > $against) {
        $row->won++;
        $row->points += self::WIN;
        $row->form[] = 'W';
    } elseif ($for === $against) {
        $row->drawn++;
        $row->points += self::DRAW;
        $row->form[] = 'D';
    } else {
        $row->lost++;
        $row->form[] = 'L';
    }
}
}
}

```

C.2 Player statistics (excerpt) (app/Services/PlayerStatisticsService.php)

```

<?php

namespace App\Services;

use App\Models\Fixture;
use App\Models\FixturePlayer;
use App\Models\MatchEvent;
use App\Models\Player;
use App\Models\Season;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Support\Collection;
use Illuminate\Support\Facades\DB;

/**
 * Player numbers are derived from match data rather than typed in:
 * - appearances: named in a lineup, or involved in an event, in a live/completed match
 * - goals / assists / cards: match events (an assist is an `assist` event or the assister
 *   on a goal)
 * - clean sheets: goalkeeper starts where the opponent did not score
 */
class PlayerStatisticsService
{
    /** Match statuses whose events count towards statistics. */
    private const COUNTED = ['live', 'completed'];

    /**
     * @return array{appearances:int, starts:int, goals:int, assists:int, yellow_cards:int,
     *   red_cards:int, clean_sheets:int}
     */
    public function forPlayer(Player $player, ?Season $season = null): array
    {
        $fixtures = $this->countedFixtures($season);
    }
}

```

```

$events = MatchEvent::query()
    ->whereIn('fixture_id', $fixtures)
    ->where(fn (Builder $q) => $q->where('player_id',
        $player->id)->orWhere('related_player_id', $player->id))
    ->get(['fixture_id', 'player_id', 'related_player_id', 'type']);

$mine = $events->where('player_id', $player->id);
$related = $events->where('related_player_id', $player->id);

$lineups = FixturePlayer::query()
    ->where('player_id', $player->id)
    ->whereIn('fixture_id', $fixtures)
    ->get(['fixture_id', 'is_starting']);

// A substitute coming on (related player on a substitution) has also appeared.
$appeared = $lineups->pluck('fixture_id')
    ->merge($mine->pluck('fixture_id'))
    ->merge($related->where('type', 'substitution')->pluck('fixture_id'))
    ->unique();

return [
    'appearances' => $appeared->count(),
    'starts' => $lineups->where('is_starting', true)->count(),
    'goals' => $mine->whereIn('type', MatchEvent::SCORING)->count(),
    'assists' => $mine->where('type', 'assist')->count() +
        $related->whereIn('type', MatchEvent::SCORING)->count(),
    'yellow_cards' => $mine->where('type', 'yellow_card')->count(),
    'red_cards' => $mine->where('type', 'red_card')->count(),
    'clean_sheets' => $player->position === 'goalkeeper' ?
        $this->cleanSheets($player, $season) : 0,
];
}

/**
 * Top scorers across counted matches.
 *
 * @return Collection<int, object{player:Player, goals:int}>
 */
public function topScorers(?Season $season = null, ?int $competitionId = null, int
    $limit = 5): Collection
{
    $rows = MatchEvent::query()
        ->whereIn('fixture_id', $this->countedFixtures($season, $competitionId))
        ->whereIn('type', MatchEvent::SCORING)
        ->whereNotNull('player_id')
        ->select('player_id', DB::raw('COUNT(*) AS goals'))
        ->groupBy('player_id')
        ->orderByDesc('goals')
        ->limit($limit)
        ->get();

    $players = Player::withTrashed()->with('team')->whereIn('id',
        $rows->pluck('player_id'))->get()->keyBy('id');

```

C.3 Administrator access middleware (app/Http/Middleware/EnsureUserIsAdmin.php)

```

<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserIsAdmin
{
    /** Only users with the admin role may reach /admin routes. */
    public function handle(Request $request, Closure $next): Response
    {
        abort_unless($request->user()?->isAdmin(), 403, 'This area is for portal
            administrators only.');
```

```

    }
}

```

C.4 Validation of match events against the fixture's teams

(app/Http/Requests/Admin/MatchEventRequest.php)

```

<?php

namespace App\Http\Requests\Admin;

use Illuminate\Foundation\Http\FormRequest;
use App\Models\MatchEvent;
use Illuminate\Validation\Rule;

class MatchEventRequest extends FormRequest
{
    /** Access is enforced by the `admin` route middleware. */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }

    public function rules(): array
    {
        $fixture = $this->route('fixture');
        $teamId = $this->integer('team_id');
        $type = $this->input('type');

        // Players must belong to the chosen team, which must be playing in this fixture.
        $teamPlayer = Rule::exists('players', 'id')->where('team_id',
            $teamId)->whereNull('deleted_at');

        return [
            'type' => ['required', Rule::in(array_keys(MatchEvent::TYPES))],
            'team_id' => ['required', 'integer', Rule::in([$fixture->home_team_id,
                $fixture->away_team_id])],
            'player_id' => ['required', 'integer', $teamPlayer],
            'related_player_id' => [
                Rule::requiredIf($type === 'substitution'),
                Rule::prohibitedIf(! in_array($type, ['goal', 'penalty_scored',
                    'substitution'], true)),
                'nullable', 'integer', 'different:player_id', $teamPlayer,
            ],
            'minute' => ['required', 'integer', 'min:0', 'max:130'],
            'additional_minute' => ['nullable', 'integer', 'min:1', 'max:30'],
            'description' => ['nullable', 'string', 'max:255'],
        ];
    }

    public function messages(): array
    {
        return [
            'team_id.in' => 'Choose one of the two teams playing in this match.',
            'player_id.exists' => 'The selected player is not in the chosen team.',
            'related_player_id.exists' => 'The second player must be in the same team.',
            'related_player_id.required' => 'Choose the player coming on.',
            'related_player_id.different' => 'The two players must be different.',
            'related_player_id.prohibited' => 'Only goals and substitutions take a second
                player.',
        ];
    }

    public function attributes(): array
    {
        return ['player_id' => 'player', 'related_player_id' => 'second player', 'team_id'
            => 'team'];
    }
}

```

C.5 Rich-text sanitisation (excerpt) (app/Support/RichText.php)

```

<?php

namespace App\Support;

use Symfony\Component\HtmlSanitizer\HtmlSanitizer;
use Symfony\Component\HtmlSanitizer\HtmlSanitizerConfig;

/**
 * Article bodies written in the admin's rich-text editor (Trix).
 * HTML is cleaned against a strict allowlist on save AND on display, so no script,
 * style, event handler or javascript: link can ever reach a visitor's browser.
 * Older plain-text articles are converted to safe HTML on display.
 */
class RichText
{
    private const MAX_INPUT = 100_000;

    private static ?HtmlSanitizer $sanitizer = null;

    public static function sanitize(string $html): string
    {
        return self::sanitizer()->sanitize($html);
    }

    /** Safe HTML for any stored article body (rich HTML or legacy plain text). */
    public static function toHtml(?string $content): string
    {
        $content = (string) $content;

        return self::isHtml($content) ? self::sanitize($content) :
            self::fromPlainText($content);
    }

    public static function isHtml(string $content): bool
    {
        return (bool) preg_match('~<(div|p|br|h[1-6]|ul|ol|li|blockquote|pre|strong|b|em|i|a|del|s)\b~i', $content);
    }

    /** True when the editor produced no actual text (e.g. "<div><br></div>"). */
    public static function isBlank(?string $html): bool
    {
        return self::plainText($html) === '';
    }

    public static function plainText(?string $html): string
    {
        $text = html_entity_decode(strip_tags(preg_replace('~<(br|div|p|h\d|li|blockquote)>~i', ' $0', (string) $html)), ENT_QUOTES | ENT_HTML5, 'UTF-8');

        return trim(preg_replace('/[\s\x{00A0}]+/u', ' ', $text));
    }

    /**
     * Legacy format: blank lines separate paragraphs, "## " starts a heading and "> " a
     * quote.
     * Every piece of text is escaped.
     */
    public static function fromPlainText(string $text): string
    {
        $blocks = preg_split("/\R{2,}/", trim($text), -1, PREG_SPLIT_NO_EMPTY);

        return collect($blocks)->map(function (string $block) {
            $block = trim($block);

```

C.6 Route definitions (routes/web.php)

```

<?php

use App\Http\Controllers\Admin;
use App\Http\Controllers>ContactController;
use App\Http\Controllers\FixtureController;

```

```

use App\Http\Controllers\FrontendController;
use App\Http\Controllers\GalleryController;
use App\Http\Controllers\HomeController;
use App\Http\Controllers\LeagueTableController;
use App\Http\Controllers\NewsController;
use App\Http\Controllers\PlayerController;
use App\Http\Controllers\ProfileController;
use App\Http\Controllers\StaffController;
use App\Http\Controllers\TeamController;
use App\Http\Controllers\TrialApplicationController;
use App\Http\Controllers\VideoController;
use Illuminate\Support\Facades\Route;

/*
|-----
| Public website
|-----
*/

Route::get('/', HomeController::class)->name('home');

// Club
Route::controller(FrontendController::class)->group(function () {
    Route::get('/about', 'about')->name('about');
    Route::get('/history', 'history')->name('history');
    Route::get('/player-development', 'academy')->name('academy');
    Route::get('/fan-zone', 'fanZone')->name('fan.zone');
    Route::get('/membership', 'membership')->name('membership');
    Route::get('/sponsors', 'sponsors')->name('sponsors');
    Route::get('/faq', 'faq')->name('faq');
    Route::get('/contact', 'contact')->name('contact');
    Route::get('/match-day', 'matchDay')->name('match.day');
    Route::get('/privacy', 'privacy')->name('privacy');
    Route::get('/terms', 'terms')->name('terms');
});

Route::post('/contact', [ContactController::class,
    'store']->middleware('throttle:5,1')->name('contact.store'));

Route::get('/management', [StaffController::class, 'management']->name('management');
Route::get('/coaching-staff', [StaffController::class, 'coaching']->name('coaching.staff');

// Teams & players
Route::get('/teams', [TeamController::class, 'index']->name('teams.index');
Route::get('/teams/{team:slug}', [TeamController::class, 'show']->name('teams.show');
Route::get('/squad', [PlayerController::class, 'index']->name('squad');
Route::get('/players/{player:slug}', [PlayerController::class,
    'show']->name('players.show');
Route::redirect('/player-profile', '/squad', 301)->name('player.profile'); // legacy static
URL

// Trials
Route::get('/join-the-team', [TrialApplicationController::class, 'create']->name('trials');
Route::post('/join-the-team', [TrialApplicationController::class, 'store'])
    ->middleware('throttle:5,1')
    ->name('trials.store');

// Fixtures, results & standings
Route::get('/fixtures', [FixtureController::class, 'index']->name('fixtures');
Route::get('/matches/{fixture}', [FixtureController::class,
    'show']->whereNumber('fixture')->name('matches.show');
Route::get('/match-details', [FixtureController::class, 'latest']->name('match.details');
    // "Match Centre": latest/next match
Route::get('/league-table', LeagueTableController::class)->name('league.table');

// News & media
Route::get('/news', [NewsController::class, 'index']->name('news');
Route::redirect('/news/article', '/news', 301)->name('news.article'); // legacy static URL
Route::get('/news/{newsPost:slug}', [NewsController::class, 'show']->name('news.show');
Route::get('/gallery', GalleryController::class)->name('gallery');
Route::get('/videos', VideoController::class)->name('videos');
/*

```

```

|-----
| Account area (Breeze)
|-----
*/

Route::get('/dashboard', function () {
    // Administrators work from the admin panel.
    return auth()->user()->isAdmin() ? redirect()->route('admin.dashboard') :
        view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class,
        'destroy'])->name('profile.destroy');
});

/*
|-----
| Admin panel – authenticated administrators only
|-----
*/

Route::middleware(['auth', 'admin'])->prefix('admin')->name('admin.')->group(function () {
    Route::redirect('/', '/admin/dashboard');
    Route::get('/dashboard', Admin\DashboardController::class)->name('dashboard');

    // Football
    Route::resource('seasons', Admin\SeasonController::class)->except('show');
    Route::resource('competitions', Admin\CompetitionController::class)->except('show');
    Route::resource('teams', Admin\TeamController::class);
    Route::resource('players', Admin\PlayerController::class)->except('show');

    // Matches
    Route::resource('fixtures', Admin\FixtureController::class);
    Route::put('fixtures/{fixture}/result', [Admin\FixtureController::class,
        'updateResult'])->name('fixtures.result');
    Route::post('fixtures/{fixture}/events', [Admin\MatchEventController::class,
        'store'])->name('fixtures.events.store');
    Route::delete('fixtures/{fixture}/events/{event}', [Admin\MatchEventController::class,
        'destroy'])->name('fixtures.events.destroy');
    Route::put('fixtures/{fixture}/lineups/{team}', [Admin\LineupController::class,
        'update'])->name('fixtures.lineups.update');
    Route::get('match-events', [Admin\MatchEventController::class,
        'index'])->name('match-events.index');

    // Content
    Route::resource('news', Admin\NewsController::class)->parameters(['news' =>
        'newsPost'])->except('show');
    Route::resource('gallery', Admin\GalleryController::class)->parameters(['gallery' =>
        'galleryItem'])->except('show');
    Route::resource('videos', Admin\VideoController::class)->except('show');

    // People
    Route::resource('staff', Admin\StaffController::class)->parameters(['staff' =>
        'staff'])->except('show');
    Route::resource('trial-applications', Admin\TrialApplicationController::class)
        ->parameters(['trial-applications' => 'trialApplication'])
        ->only(['index', 'show', 'update', 'destroy']);
    Route::resource('contact-messages', Admin\ContactMessageController::class)
        ->parameters(['contact-messages' => 'contactMessage'])
        ->only(['index', 'show', 'update', 'destroy']);
});

require __DIR__.'/auth.php';

```